TVM 기반 LLM NPU compiler

계산의 의미, 실행 배치, 메모리와 검증을 실제 예제로 연결합니다.

동작을 따라 읽는 그림

한 모델이 실행되는 세 가지 관점

모델의 계산
X → attention → MLP
28 layers → logits

→

실행 배치
64×64 tile + panel
FP32 누산 → FP16 저장

→

값의 보관
SRAM 재사용 / DRAM
KV cache와 주소 계약

IR와 명령, 실험 결과로 선택을 확인
작은 예제: 실제 shape·수명·주소를 읽기
Llama: hidden 3072 / MLP 8192 / Q 24 heads / KV 8 heads

01 설명의 출발점

모델이 정의한 계산을 NPU에서 실행하려면 수식, 실행 순서, 메모리 주소를 모두 정해야 합니다. 이 발표는 같은 예제를 이 세 관점으로 이어서 설명합니다.

02 두 가지 예제를 함께 사용

작은 Linear·두 matmul 예제로 IR과 주소를 읽고, Llama의 실제 hidden 3072·MLP 8192 크기로 확장합니다. 작은 그림이 실제 모델의 어떤 부분인지 연결합니다.

03 현재 기준

메모리 계층이 적용된 v09 FP16 backend가 본문입니다. 양자화와 기존 main은 부록으로 구분하며, 실험 수치는 문서에 남아 있는 검증 기록을 사용합니다.

예제와 해석

읽는 순서: 모델 → Relax/TIR → fusion → schedule → 메모리 → ISA → runtime → 검증

세 가지 질문으로 compiler를 이해합니다

같은 계산이라도 실행 순서와 저장 위치에 따라 비용이 달라집니다.

동작을 따라 읽는 그림

단계	같은 예제가 어떻게 보이는가	이번 단계에서 정하는 것
계산 의미	Y = SiLU(X × B + bias)	연산 연결 · reduction · cast
실행 배치	출력 64×64 / K를 64씩 2번	Loop 순서 · cache 위치
저장과 주소	입력 X/B 각 16 KiB를 SRAM에	수명 · 재사용 · 실제 offset
실행과 검증	M_MUL + vector + DMA stream	ABI · 출력 비교 · 비용

같은 계산을 단계별로 구체화

01 계산: 값의 의미를 정한다

Relax는 matmul·add·SiLU의 연결을 표현합니다. Semantic TIR는 각 원소의 수식, reduction 축, FP16/FP32 변환 위치를 정의합니다.

02 배치: 같은 계산을 나누어 실행한다

Schedule은 loop를 64 단위로 나누고 입력 panel을 어느 반복 밖에서 올릴지 정합니다. 같은 tensor도 재읽는 횟수에 따라 DMA가 달라집니다.

03 보관: 다음 계산까지 이어준다

Memory planning은 값의 마지막 사용과 작업 공간을 함께 봅니다. 두 커널을 합치지 않아도 같은 SRAM 영역으로 중간값을 넘길 수 있습니다.

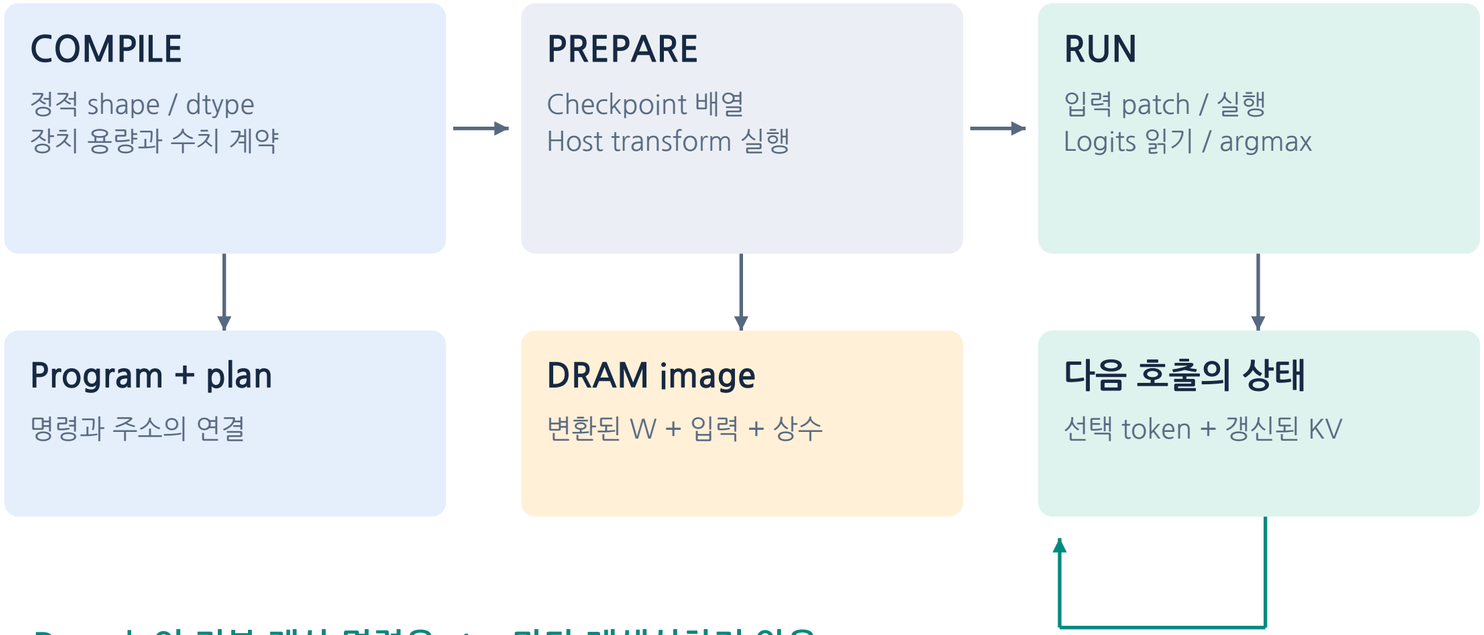
예제와 해석

관통 예제: Y = SiLU(X[64,128] × B[128,64] + bias[64]). 세 단계는 모두 이 계산을 구체화합니다.

Compile, 준비, 실행은 서로 다른 작업입니다

프로그램을 만드는 시점과 weight를 실제로 읽는 시점을 구분합니다.

동작을 따라 읽는 그림



Decode의 기본 계산 명령은 step마다 재생성하지 않음

01 Compile: 정적 구조를 결정

Shape·dtype·장치 조건을 받아 계산 프로그램과 주소 계획을 만듭니다. 값이 parameter라면 실제 weight 내용은 준비 단계에 넣을 수 있습니다.

02 Prepare: 실제 배열을 배치

Host transform을 실행하여 W.T 등을 만들고, 변환된 signature 순서대로 DRAM image를 채웁니다. 이 준비 비용과 NPU 실행 비용을 구분합니다.

03 Run: 바뀐 입력을 넣어 실행

Decode는 token·position·mask와 KV 상태를 갱신하며 기본 계산 프로그램을 재사용합니다. Constant로 접힌 weight까지 임의로 바꿀 수 있다는 뜻은 아닙니다.

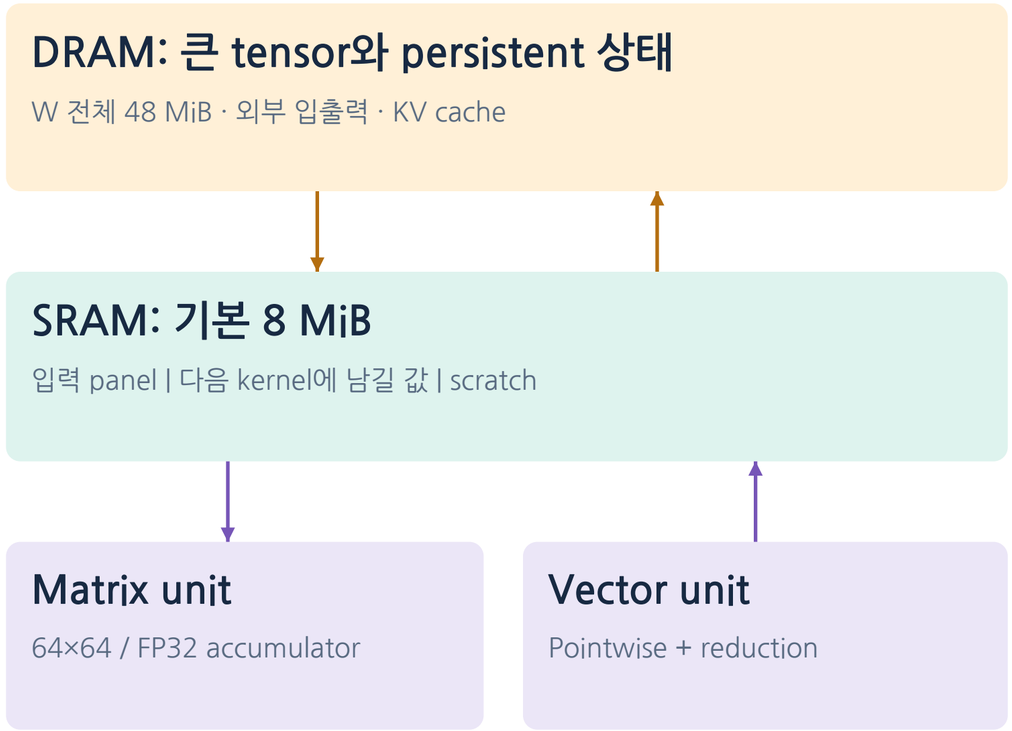
예제와 해석

예: transpose 함수는 compile 때 만들어지지만, 실제 weight 배열의 transpose는 host 준비 때 실행합니다.

장치는 세 층의 메모리를 사용합니다

GLOAD/GSTORE와 LOAD/SAVE는 서로 다른 경계를 이동합니다.

동작을 따라 읽는 그림



GLOAD / GSTORE

LOAD / SAVE

SRAM 재사용은 DRAM 이동을 줄임. Unit LOAD는 여전히 필요.

01 DRAM: 모델과 호출 간 상태

Weight 전체, 외부 입력·출력, persistent KV cache가 놓입니다. 기본 SRAM 8 MiB보다 큰 tensor는 계산에 필요한 부분만 가져옵니다.

02 SRAM: 계산할 조각과 보관할 값

입력 panel, vector scratch, 다음 커널이 쓸 중간값이 공간을 나눠 씁니다. 보관을 늘리면 현재 커널에 줄 수 있는 작업 공간은 줄어듭니다.

03 Unit: 실제 계산과 누산 상태

Matrix unit의 FP32 accumulator는 SRAM buffer와 다릅니다. SRAM에 값이 있어도 연산장치 입력으로 LOAD하고, 완료된 결과를 SAVE하는 단계는 필요합니다.

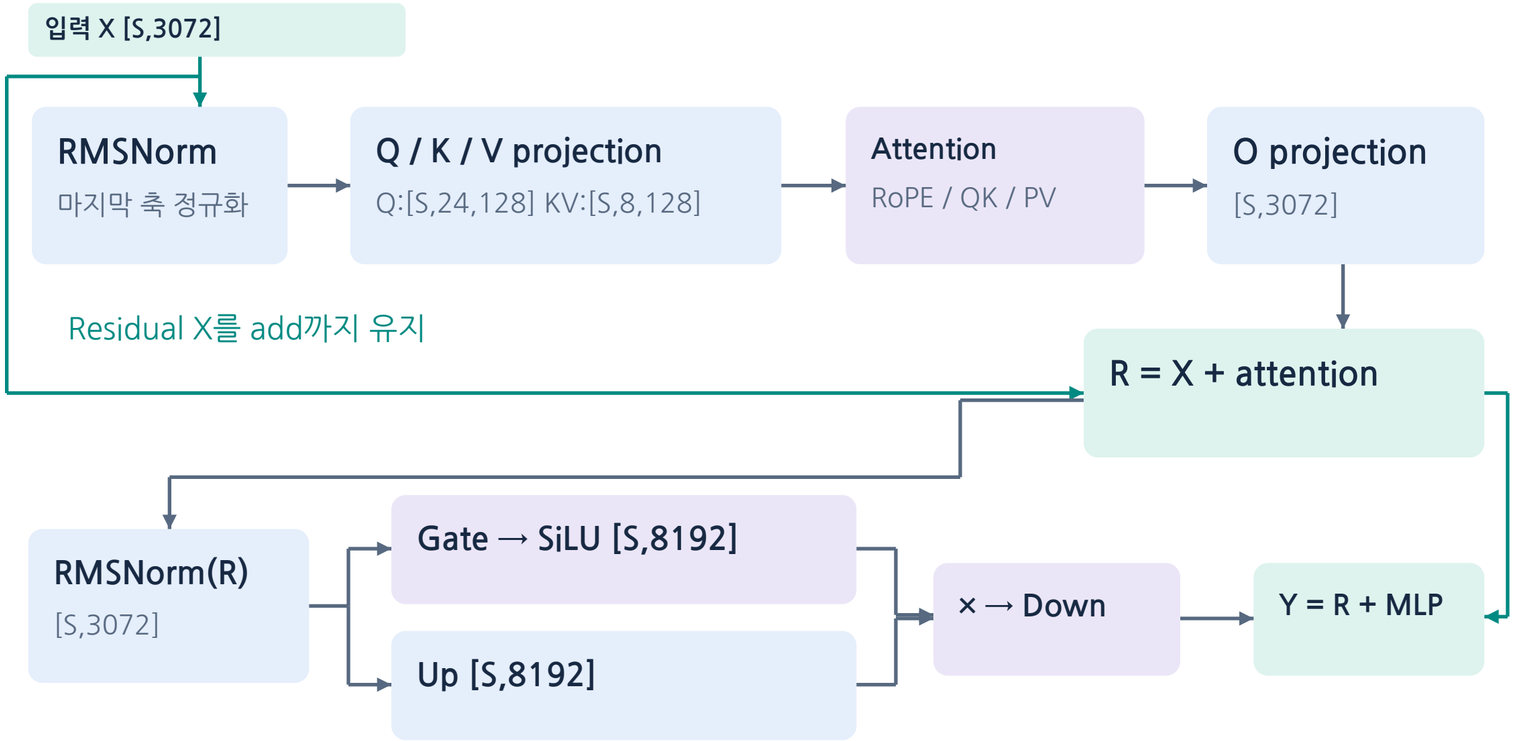
예제와 해석

GLOAD/GSTORE는 DRAM↔SRAM, LOAD/SAVE는 SRAM↔연산장치 이동입니다. 이 둘을 섞어 세지 않습니다.

Llama 한 layer의 계산을 끝까지 따라갑니다

Llama 3.2 3B: hidden 3072 · MLP 8192 · Q 24 heads / KV 8 heads

동작을 따라 읽는 그림



01 Attention의 입력과 출력

X[S,3072]를 정규화하고 Q/K/V를 만듭니다. Q는 24 heads, K/V는 8 heads, 각 head 폭은 128이며 Q/K에 RoPE를 적용합니다.

02 MLP는 두 갈래 계산

Gate와 up은 각각 3072→8192 projection입니다. SiLU(gate)와 up을 원소별로 곱한 뒤 down으로 8192→3072를 계산합니다.

03 Residual은 수명을 늘린다

입력 X는 attention이 끝난 뒤 add할 때까지 필요합니다. 첫 residual 결과 R도 MLP 뒤에 다시 쓰므로, 바로 다음 연산만 보고 공간을 해제하면 안 됩니다.

예제와 해석

S=64이면 hidden activation은 384 KiB, gate/up 출력은 각각 1 MiB입니다. 전체 모델은 이 layer를 28번 사용합니다.

TVM의 표준 변환 위에 NPU의 결정을 얹습니다

TVM은 IR·변환·분석을 제공하고 backend는 장치 계약과 선택 기준을 제공합니다.

동작을 따라 읽는 그림

단계	TVM 표준 기반	NPU가 추가하는 결정
그래프	Bindings / CSE / LiftTransformParams	Attention 의미·지원 layout
계산 정의	LegalizeOps / TE	FP32 누산과 FP16 저장
Fusion	Annotate / FuseOps / FuseTIR	지원 영역 / 부분·전체 비용
Schedule	split / reorder / cache / tensorize	Panel·복사 위치·장치 intrinsic
Memory	StaticPlanBlockMemory / USMP	호출 간 보관 + schedule 재선택
실행	IR 분석 기반 codegen 연결	ISA / linker / runtime ABI

01 표준 TVM이 제공하는 도구

LegalizeOps·FuseOps/FuseTIR가 계산을 변환하고, Schedule API가 loop와 cache를 바꿉니다. StaticPlanBlockMemory·USMP는 메모리 분석에 사용합니다.

02 NPU backend가 정하는 정책

FP32 누산·FP16 저장 계약, 64×64 intrinsic, 후보 탐색, SRAM 보관, ISA 인코딩은 장치 특성에 맞게 구현합니다. 표준 pass 호출만으로 정해지지 않습니다.

03 선택은 후단 결과까지 본다

후보를 schedule·메모리 배치·명령 생성까지 내려 DMA와 words를 비교합니다. 따라서 파이프라인은 변환의 순서와 후보를 되돌아보는 선택 과정을 함께 가집니다.

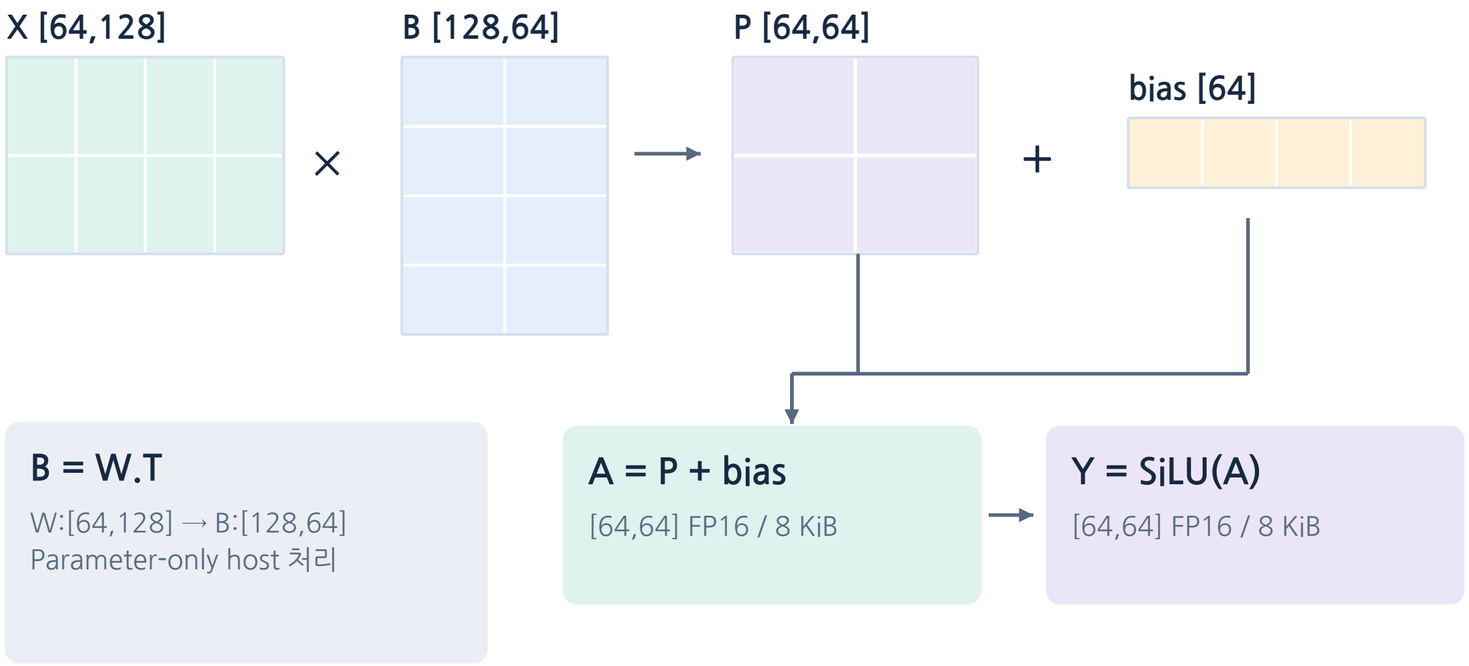
예제와 해석

DLight를 직접 호출하지 않지만 TVM Schedule API를 사용합니다. API와 그 위의 선택 정책을 구분해서 읽습니다.

작은 예제 하나로 중간 결과를 연결합니다

실제 Llama에는 없는 bias를 추가해 broadcast와 fusion까지 살펴봅니다.

동작을 따라 읽는 그림



01 모델 입력의 의미

X는 64개 위치의 128차원 입력이고 W는 출력 채널 64개에 대한 weight입니다. Host에서 B=W.T[128,64]를 준비합니다.

02 중간 tensor의 의미

Matmul 출력 P와 add 출력 A는 각각 [64,64] FP16, 8 KiB입니다. Bias[64]는 64개 행에 broadcast되어 같은 열끼리 더해집니다.

03 관찰할 파일을 연결

Authored Relax에서는 연산 연결을, semantic TIR에서는 수식을, scheduled TIR에서는 cache와 tile을 봅니다. 최종 program.json에서 실제 비용을 확인합니다.

예제와 해석

작은 예제의 K=128은 64개씩 두 번 누산합니다. Matmul→add→SiLU의 FP16 저장 경계는 fusion 뒤에도 유지합니다.

Weight에만 의존하는 계산은 host로 옮깁니다

LiftTransformParams는 함수를 분리하고 runtime 준비 단계가 실제 배열을 변환합니다.

동작을 따라 읽는 그림

변환 전: 매 호출 graph

```
device(x, W, bias)
  B = transpose(W)
  y = SiLU(x @ B + bias)
```

LiftTransformParams 이후

```
host_transform(W, bias)
  return (bias, transpose(W))
```

Device 입력 signature

```
device(x, bias, B)
  y = SiLU(x @ B + bias)
```

실제 예제에서 달라진 순서

원래: x / W / bias
변환 후: x / bias / lv (= B)
Runtime이 새 signature에 맞춰 연결

01 분리 기준은 parameter 의존성

Transpose가 W에만 의존하면 실행 입력 x가 달라져도 결과는 같습니다. LiftTransformParams가 이 계산을 host 함수로 분리합니다.

02 함수의 실제 입력은 ABI로 확인

Device에는 x와 변환된 parameter가 들어갑니다. 작은 예제의 순서는 x, bias, lv(B)이며 원래 W·bias 순서를 그대로 가정하면 틀립니다.

03 장치 메모리 배치와는 별개

Host가 W.T 전체를 만들었다고 SRAM에 전체가 올라가는 것은 아닙니다. 준비된 B는 DRAM에 있고, schedule이 필요한 panel을 가져옵니다.

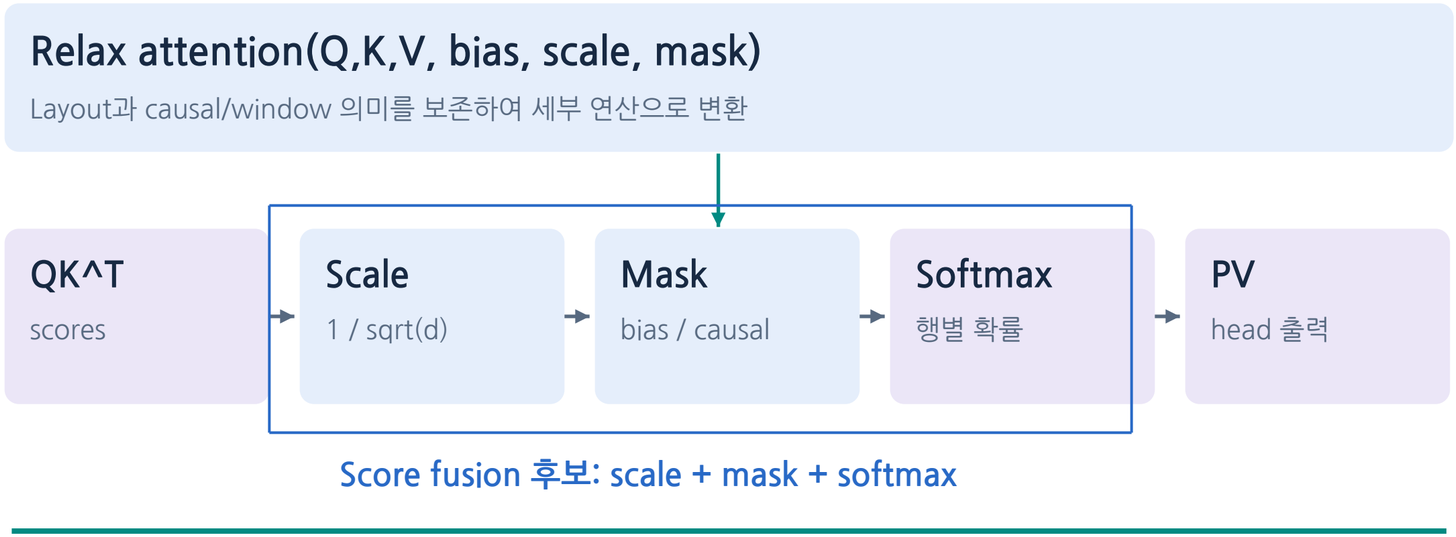
예제와 해석

Llama LM head W는 약 751.5 MiB입니다. Parameter-only 변환을 token마다 반복하지 않는 것이 중요합니다.

Attention을 세부 연산으로 드러내는 이유가 있습니다

CanonicalizeAttention은 지원되는 attention 의미를 Relax의 계산 연결로 바꿉니다.

동작을 따라 읽는 그림



분해 = 수식 공개 / Fusion = 함께 실행할 영역 선택

01 고수준 attention 안을 드러낸다

CanonicalizeAttention은 QK^T→scale/mask→softmax→PV를 Relax에 나타냅니다. Layout·bias·causal/window 의미를 보존하는 NPU용 변환입니다.

02 NPU용 수치 정의를 적용한다

표준 attention legalizer 내부 TE가 우리 matmul·softmax legalizer를 다시 호출한다고 보장할 수 없습니다. 먼저 연산을 드러내 각 정의를 적용합니다.

03 분해 이후 다시 합칠 수 있다

세부 연산을 보이게 만드는 단계와 하나의 kernel로 실행하는 선택은 다릅니다. Score 또는 full-attention fusion 후보가 뒤에서 만들어질 수 있습니다.

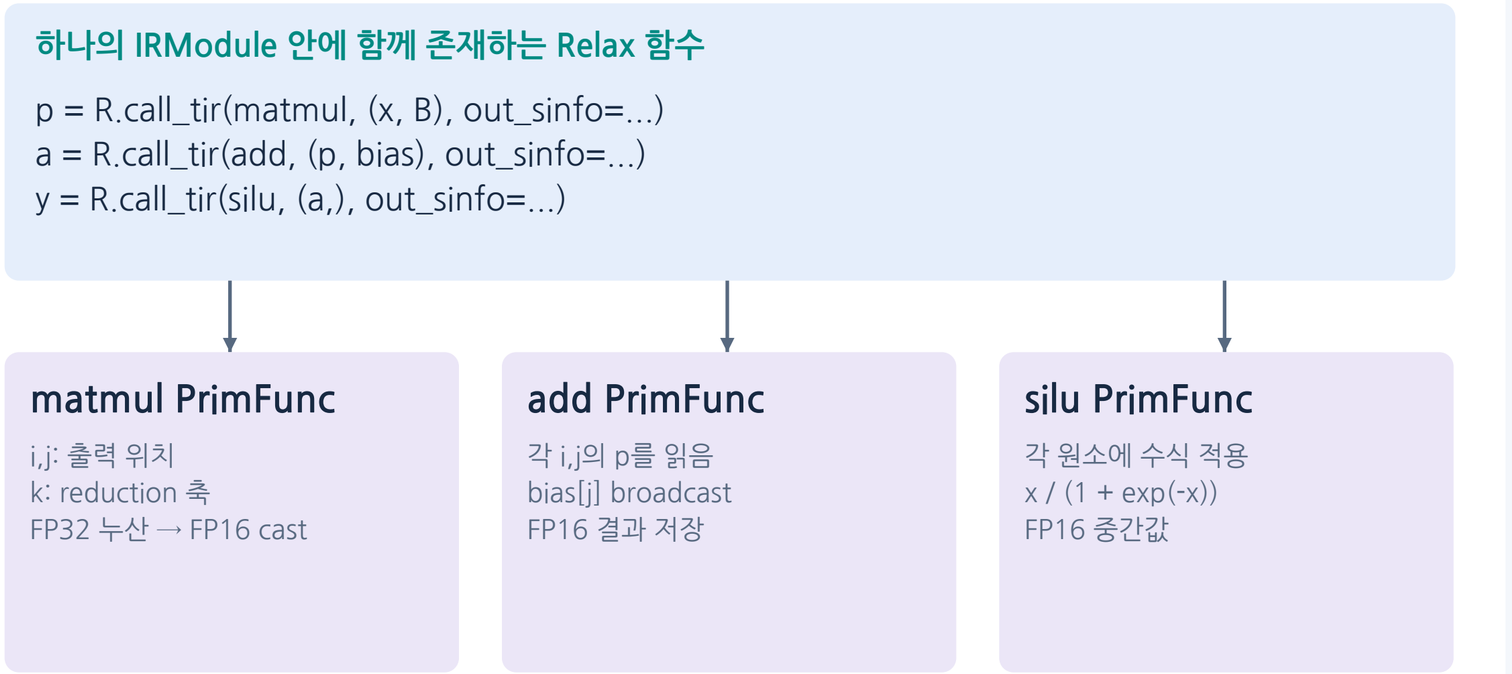
예제와 해석

현재 Llama authoring은 이미 세부 attention 연산을 사용합니다. 그 부분에서는 이 pass가 새로 분해할 것이 없을 수 있습니다.

LegalizeOps 뒤에도 Relax는 남습니다

Relax는 호출 관계를 유지하고 TIR는 각 호출의 원소 계산을 정의합니다.

동작을 따라 읽는 그림



01 Relax는 호출 관계로 남는다

LegalizeOps는 고수준 연산을 call_tir 호출로 바꾸고 계산을 TIR PrimFunc에 정의합니다. Relax 함수 전체가 사라지는 것이 아닙니다.

02 TIR는 원소 계산을 표현한다

Matmul의 i,j는 결과 위치이고 k는 합산 축입니다. T.init은 초기값, reads/writes는 접근 영역을 나타내어 후속 변환의 합법성 분석에 쓰입니다.

03 TE는 이 정의를 만드는 도구

placeholder·compute·reduce_axis로 수식을 적고 bb.call_te로 PrimFunc와 Relax 호출을 만듭니다. TE가 직접 ISA를 실행하는 것은 아닙니다.

예제와 해석

그래프를 묶는 FuseOps가 legalization 뒤에도 가능한 이유: 호출 연결은 Relax에, 각 호출의 loop는 TIR에 함께 남아 있습니다.

FP32 누산은 전체 K가 끝날 때까지 이어집니다

X[64,128] @ B[128,64]: 출력 tile 하나에 K tile 두 개를 사용합니다.

동작을 따라 읽는 그림



01 K 조각은 같은 출력에 더한다

X[:,0:64]와 X[:,64:128]은 같은 64×64 출력 타일의 서로 다른 합산 항입니다. 두 결과를 같은 FP32 accumulator에 이어서 더합니다.

02 SAVE는 그 타일의 K가 끝난 뒤

첫 matrix 명령은 MAC=0, 이후 K 조각은 MAC=1입니다. 마지막 K를 마친 값만 FP16으로 SAVE하며 중간 FP16 저장은 수치 의미를 바꿉니다.

03 Activation은 현재 자동 사용 안 함

Activation을 붙인다면 해당 출력의 마지막 K 이후, bias 및 반올림 순서까지 맞아야 합니다. 현재 자동 matmul 경로는 activation OFF이고 SiLU는 vector 계산입니다.

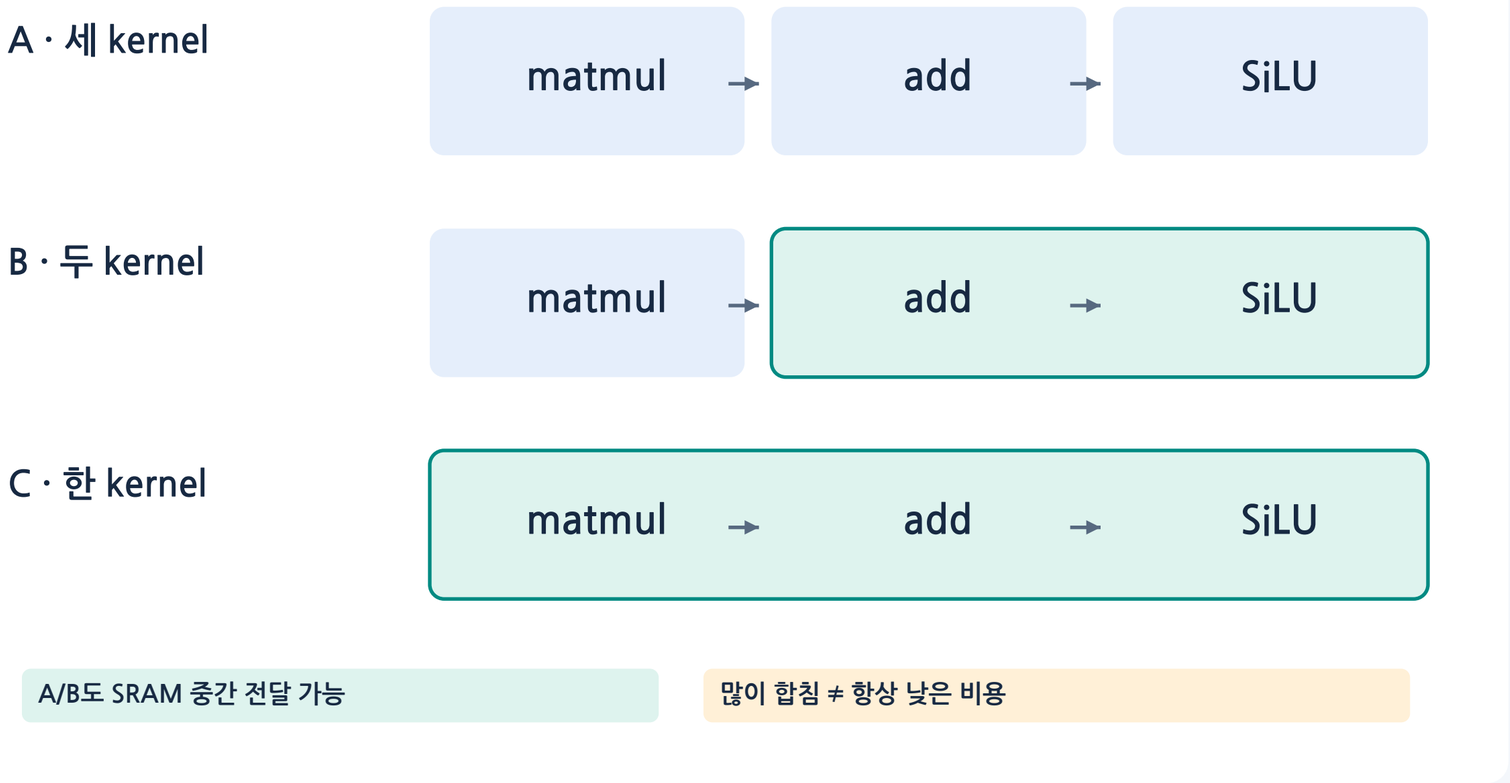
예제와 해석

출력 타일이 바뀌면 새 누산을 시작합니다. 모든 하드웨어에서 K가 항상 최내측이어야 한다는 일반 법칙은 아닙니다.

커널 수가 줄어드는 것만으로 이득을 판단하지 않습니다

분리된 커널도 SRAM에서 중간값을 전달할 수 있습니다.

동작을 따라 읽는 그림



01 Fusion은 함께 배치할 기회를 준다

Matmul 뒤 add/SiLU를 같은 PrimFunc에 넣으면 방금 만든 타일 가까이 소비 계산을 배치할 수 있습니다. Buffer와 명령 설정도 줄어들 수 있습니다.

02 분리 kernel도 SRAM으로 연결 가능

Memory planning이 중간값 전체를 보관하면 분리된 kernel 사이 GSTORE/GLOAD도 제거됩니다. 커널 경계가 곧 DRAM 왕복이라는 뜻은 아닙니다.

03 최종 비용으로 판단한다

Fusion이 scratch를 늘리거나 짧은 vector 명령을 많이 만들면 불리할 수 있습니다. DMA가 같을 때 ISA words와 SRAM peak까지 확인합니다.

예제와 해석

A/B/C는 가능한 개념 구성입니다. 현재 탐색이 모든 부분 fusion 조합을 빠짐없이 생성한다는 뜻은 아닙니다.

연산의 성격과 모든 연결 경로를 보고 묶습니다

Pattern 번호가 같아서 합치는 것이 아니라 graph의 의존성과 외부 사용을 검사합니다.

동작을 따라 읽는 그림



분류	예	Fusion에서 확인할 것
0 / 1 / 2	원소별 / broadcast / injective	연결 경로와 같은 group의 제약
3 / 4	Reduction / matmul 계열	임의의 복합 연산끼리 합치지 않음
7 / 8	Tuple / opaque	Tuple 관계 / 일반 fusion 경계

01 기본 분류는 계산의 성격

Elemwise=0, broadcast=1, injective=2, reduce=3, out-elementwise-fusable=4, tuple=7, opaque=8입니다. 번호는 성능 점수도 연속된 등급도 아닙니다.

02 분기에서는 중간 경로도 검사

Matmul 결과가 여러 소비자로 갈라지면 한 경로만 보고 묶지 않습니다. 다시 만나는 지점과 관련 경로의 pattern, 외부 출력·사용을 함께 확인합니다.

03 장치에서의 이득은 후속 판단

NPU는 미지원 영역을 opaque로 보호할 수 있습니다. FuseOps가 만든 그룹을 FuseTIR로 결합한 뒤 실제 schedule과 비용으로 채택 여부를 정합니다.

예제와 해석

Attention pattern은 Relax 연산이 보일 때 먼저 묶을 수 있고, 주변의 일반 fusion과 같은 후보 안에서 함께 존재할 수 있습니다.

영역 하나를 검사하고, 전체 프로그램에서 다시 확인합니다

부분 검사와 전체 검사는 서로 다른 질문에 답합니다.

동작을 따라 읽는 그림

영역 하나 검사

TVM이 만든 작은 범위
fused ↔ unfused 비교

→

전체 후보 조립

살아남은 영역 + 주변 graph
prefill / decode entry

→

전체 module 선택

보관 + 실제 주소 + 최종 명령
모든 entry 비용 확인

부분 검사도 codegen까지

전체 수명과 ABI까지

비용 비교의 우선순위

1. DMA bytes가 작으면 우선
2. DMA가 같으면 ISA words, 다시 같으면 SRAM peak

Prefill/decode 중 어떤 entry도 나빠지지 않는 개선을 채택

01 영역 검사: 합칠 가치가 있는가

npu_fusion.select는 TVM이 만든 영역 하나를 fused/unfused로 내려 비교합니다. 분리 기준에도 SRAM 전달을 적용하지만 주변 전체 수명은 아직 모릅니다.

02 전체 비교: 주변까지 넣어도 좋은가

lower_and_select는 entry 전체의 보관·주소·최종 명령을 비교합니다. Prefill/decode는 ABI가 맞는 하나의 모듈로 선택하며 entry마다 임의로 섞지 않습니다.

03 중복은 줄이고 탐색은 제한한다

영역·entry 비용 cache를 사용합니다.
DMA→words→SRAM 순서로 비교하고, 모든 entry가 나빠지지 않는 개선을 채택합니다. 모든 조합의 전역 최적을 보장하지 않습니다.

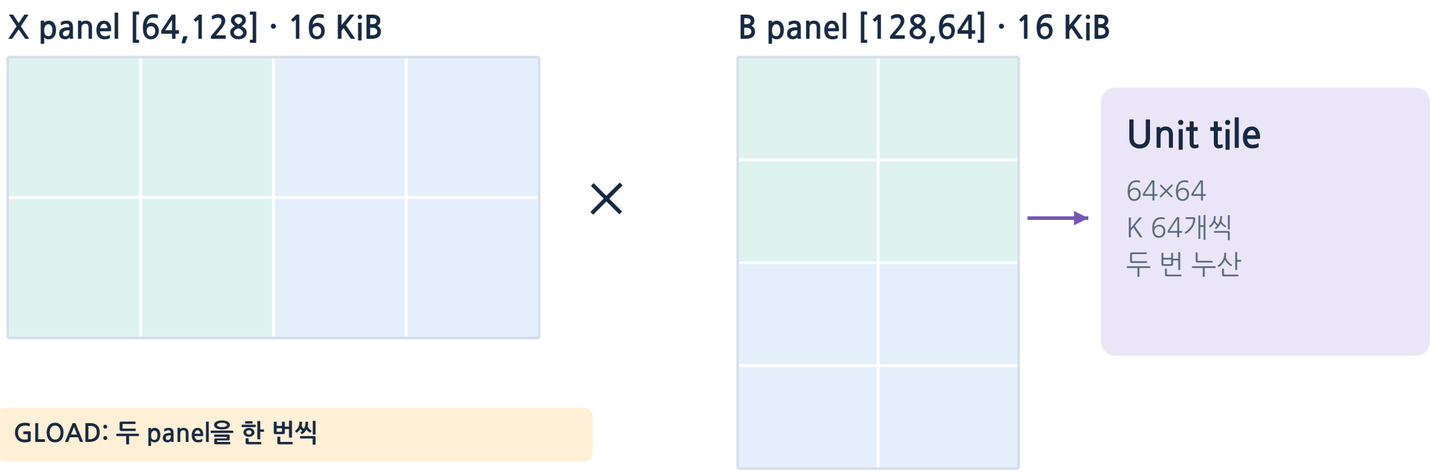
예제와 해석

부분 검사에서 버린 fusion을 모든 다른 선택과 다시 조합해 검사하지는 않습니다. 현재 탐색 범위의 한계입니다.

연산 tile과 DMA panel은 크기가 다를 수 있습니다

64×64는 matrix 명령의 단위이며, 한 번 가져올 weight 크기를 뜻하지 않습니다.

동작을 따라 읽는 그림



K loop 안에서는 SRAM의 다른 부분을 LOAD

DRAM panel 복사 위치와 연산 tile의 반복 위치를 구분

01 Tile은 연산장치가 계산하는 단위

Matrix 한 번은 64×64 크기의 계산 조각을 처리합니다. K=128이면 같은 출력 타일에 두 번의 K 계산이 필요합니다.

02 Panel은 SRAM에서 재사용하는 범위

X[64,128]과 B[128,64]를 각각 16 KiB씩 미리 가져올 수 있습니다. 두 matrix 명령은 같은 panel의 다른 절반을 LOAD합니다.

03 복사 위치가 재읽기 횟수를 정한다

K loop 바깥에서 panel을 올리면 K가 바뀔 때마다 DRAM에 가지 않습니다. k_chunk=64여도 cache 위치에 따라 K 전체가 SRAM에 있을 수 있습니다.

예제와 해석

실제 작은 예제: m_panel=n_panel=k_chunk=64, a_at=b_at=m_panel. K 반복 바깥에서 X와 B를 한 번 복사합니다.

compute_at은 필요한 계산을 사용 위치로 옮깁니다

B = A + 1, C = B × 2를 2행씩 처리하는 교육 예제입니다.

동작을 따라 읽는 그림

변환 전: B 전체를 먼저 생성

```
for i in 0..3:
  B[i,:] = A[i,:] + 1

for i in 0..3:
  C[i,:] = B[i,:] * 2
```

compute_at 뒤: 필요한 두 행

```
for io in [0, 2]:
  B[io:io+2,:] = A[...] + 1
  C[io:io+2,:] = B[...] * 2
```

B가 필요한 범위

B가 필요한 범위

2행씩

01 문제: 중간값 전체를 먼저 만든다

B=A+1, C=B×2에서 B 전체를 만든 뒤 C를 계산하면 B 전체가 한동안 필요합니다. 사용하는 행 묶음 가까이 B 계산을 옮길 수 있습니다.

02 compute_at이 바꾸는 위치

C의 행 loop를 나눈 후 B block을 그 loop 안으로 이동합니다. 매 반복에서 필요한 B의 두 행을 만들고 즉시 C 두 행을 계산합니다.

03 결과에는 API 이름이 남지 않는다

compute_at은 IR을 고치는 compiler API입니다. TIR 결과에는 이동한 loop와 block이 나타나고, 필요한 영역 축소는 후속 compaction이 반영합니다.

예제와 해석

여기서는 4×4 CPU 교육 예제의 2행 묶음을 사용합니다. 실제 NPU matrix tile이 2라는 뜻은 아닙니다.

reverse_compute_at은 소비 계산을 생산 위치에 맞춥니다

두 API 모두 loop 배치를 바꾸지만, 기준으로 삼는 쪽이 다릅니다.

동작을 따라 읽는 그림

compute_at(B, C의 loop)

C의 소비 loop가 기준

필요한 행/타일의 범위
의존성을 유지하며 배치

→

B 생산을 loop 안으로

for tile:
B tile 생성 → C tile 소비

reverse_compute_at(C, B의 loop)

B의 생산 loop가 기준

필요한 행/타일의 범위
의존성을 유지하며 배치

→

C 소비를 loop 안으로

for tile:
B tile 생성 → C tile 소비

01 compute_at: 생산자를 옮긴다

C가 사용하는 loop 안으로 B의 생산을 가져옵니다. 소비하는 범위에 맞춰 필요한 B를 만들며 중간값 전체의 보관을 줄일 수 있습니다.

02 reverse_compute_at: 소비자를 옮긴다

B를 만드는 loop 안으로 C의 소비를 가져옵니다. 생산 타일이 끝난 직후 add/SiLU나 출력 복사 같은 다음 계산을 배치할 때 사용합니다.

03 아무 위치로 이동할 수는 없다

데이터 의존성·block 범위·소비 관계가 변환을 허용해야 합니다. Matmul의 뒤 연산도 해당 출력 타일의 K 누산이 끝난 뒤 실행되어야 합니다.

예제와 해석

역방향이라는 이름은 데이터를 거꾸로 계산한다는 뜻이 아닙니다. 어느 계산의 loop에 다른 계산을 맞추는지의 차이입니다.

복사 stage, 실행 위치, buffer 크기는 별개의 결정입니다

cache_read → compute_at → CompactBufferAllocation을 구분해서 읽습니다.

동작을 따라 읽는 그림

cache_read

입력 → 새 cache buffer
소비자는 cache를 읽음

compute_at

복사를 행 묶음 안으로
매번 필요한 범위만 접근

Compaction

실제 allocation 축소
4×4 → 2×4 예제



set_scope = 기존 buffer의 scope

cache_write = 출력 cache + 복사

01 cache_read는 복사를 추가

기존 입력을 cache buffer에 복사하고 소비자가 cache를 읽도록 바꿉니다. 어느 반복에 복사할지는 compute_at으로 별도 지정합니다.

02 Compaction은 필요한 범위로 축소

4×4 교육 예제에서 매번 두 행만 접근하면 cache allocation을 2×4로 줄일 수 있습니다. API 호출 직후 보이는 논리 shape와 최종 allocation은 다를 수 있습니다.

03 set_scope와 cache_write도 다르다

set_scope는 기존 buffer의 저장 scope를 바꿉니다. cache_write는 계산이 cache에 쓰고 원래 출력으로 복사하는 stage를 만듭니다. 둘을 같은 API로 읽지 않습니다.

예제와 해석

FP32 accumulator는 unit 내부 상태로 내릴 수 있습니다. Scope 변경만 보고 전체 FP32 행렬을 SRAM에 저장한다고 판단하지 않습니다.

현재 schedule은 여러 선택 축을 조합합니다

기준 schedule도 남아 있지만 네 가지 고정 규칙만 고르는 구조는 아닙니다.

동작을 따라 읽는 그림

선택 축	후보의 예 / 뜻
m_panel / n_panel	출력 행·열 묶음: N=256이면 64 / 128 / 256
k_chunk	K를 묶는 단위; 안에서는 64개씩 matrix 실행
outer / inner order	M/N panel 순서와 panel 안 M/N tile 순서
a_at / b_at	X/W 복사를 둘 loop 위치를 각각 선택

추정 DMA 순위 → Schedule 생성 → 공간·누산 검사 → 최종 비용 비교

보관값이 차지한 공간을 제외 → 남은 SRAM으로 다시 후보 평가

01 서로 독립적인 선택 축을 조합

M/N panel, K chunk, 바깥·안쪽 M/N 순서와 X/W cache 위치를 조합합니다. Factor는 padding한 축 길이의 64배수 약수에서 만듭니다.

02 중복과 확실한 SRAM 초과 제거

실제 tile 순회·복사 주기가 같은 후보를 합칩니다. 정렬된 직접 입력은 두 operand cache만으로 용량 초과를 증명할 때 제외합니다. 나머지는 실제 SRAM 배치와 명령으로 검사합니다.

03 탐색 예산과 기록 재사용

기본 성공 목표 8개·시도 64회이며 words 한도는 커널 크기에 맞춰 늘립니다. 선택형 MetaSchedule은 ReplayFunc 탐색과 trace 저장·재생을 지원합니다.

예제와 해석

기존 기준 schedule도 비교용으로 유지합니다. 네 가지 고정 규칙만 고르는 구조도, 모든 가능한 조합을 끝까지 검사하는 구조도 아닙니다.

48 MiB weight를 SRAM에 한 번에 넣지 않습니다

64-token gate projection: X[64,3072] @ W[3072,8192]

동작을 따라 읽는 그림

X [64,3072] 384 KiB

X 유지

W [3072,8192] 전체 48 MiB

64열 panel 384 KiB씩 교체 × 128회

Gate / up의 matrix 명령

출력 tile 128개 × K tile 48개
= 6144개 M_MUL

Down의 기록된 선택

W 4 MiB panel에 출력 tile 4개
12 panels × 4 × 128 = 6144

01 Gate/up은 작은 X를 오래 쓴다

M=64에서 X[64,3072]는 384 KiB입니다. 이를 유지하고 W[3072,64] 384 KiB panel을 출력 열마다 바꾸는 선택이 기록되어 있습니다.

02 한 출력 타일은 48번 누산

K=3072이므로 3072/64=48개의 matrix 명령을 수행합니다. N=8192의 출력 열 타일 128개를 계산해 gate 전체에 6144개가 필요합니다.

03 Down은 다른 panel을 사용한다

기록된 down은 W[8192,256] 4 MiB를 유지하며 출력 64열 타일 네 개를 만듭니다. H 입력은 앞 kernel에서 SRAM에 남겨 전달합니다.

예제와 해석

이 수치는 실제 폭 MLP의 정적 compile 기록입니다. 선택 순서는 gate→SiLU→up→multiply→down이며 전체 checkpoint 실행 검증과 구분합니다.

SiLU 하나도 여러 vector 계산으로 펼쳐집니다

현재 경로는 중간 FP16 값과 SRAM 작업 공간을 사용합니다.

동작을 따라 읽는 그림



각 단계: SRAM에서 LOAD → vector 계산 → SRAM에 SAVE

임시값 수명·길이에 맞춰 scratch를 재사용하고 연속 축은 길게 처리

01 SiLU는 다섯 단계의 수식

-x, exp, 1 더하기, 역수, x와 곱하기로 펼쳐집니다. SiLU 하나를 ISA 명령 하나로 세거나 matmul activation bit 사용으로 이해하면 안 됩니다.

02 중간값도 공간과 이동이 필요

기본 FP16 vector 경로는 중간 결과를 SRAM scratch에 저장합니다. 같은 입력 x가 처음과 마지막에 필요하므로 그 수명과 cache 재사용을 함께 봅니다.

03 연속 원소를 길게 처리한다

가능한 pointwise 축을 fuse/split하여 짧은 vector 명령을 줄입니다. 비연속 stride·broadcast·겹치는 입출력은 검사하며 reduction 축을 임의로 합치지는 않습니다.

예제와 해석

SRAM에 있는 입력은 LOAD→vector 연산→SAVE로 처리합니다. 이 이동은 DRAM DMA와 다른 비용입니다.

256 lanes와 명령이 지정하는 VLEN을 구분합니다

현재 compiler/C-model 계약과 실제 RTL에서 확인할 항목을 함께 표시합니다.

동작을 따라 읽는 그림

논리 VLEN = 1000개 원소

256개	256개	256개	232개
물리 폭 단위	물리 폭 단위	물리 폭 단위	유효 tail
0..255	256..511	512..767	768..999

현재 계약의 전제와 아직 확인할 것

Compiler/C-model: 긴 논리 길이 명령을 표현
RTL 확인 필요: 256-lane 내부 반복, tail, LOAD/SAVE와 reduction 누적
명령 1개라도 처리 길이에 따라 실제 실행 cycle은 달라질 수 있음

01 VLEN은 이번 논리 처리 길이

예를 들어 길이 1000의 원소별 연산을 지정할 수 있습니다. 물리 폭이 256이면 내부적으로 256+256+256+232로 처리한다는 계약을 전제로 합니다.

02 항상 1024로 padding하지 않는다

유효 길이를 처리할 수 있다면 tail 232개만 계산하면 됩니다. 하지만 C-model이 배열로 길이 전체를 처리하는 사실은 실제 256-lane 하드웨어 구현의 증거가 아닙니다.

03 성능과 reduction은 별도 문제

VLEN=1024 한 명령과 VLEN=256 한 명령의 cycle이 같지는 않습니다. 행별 reduction을 단순히 256개씩 나누려면 부분 max/sum을 이어가는 설계도 필요합니다.

예제와 해석

긴 VLEN의 RTL LOAD/연산/SAVE·tail 계약은 후속 확인 사항입니다. 현재 ISA words만으로 lane 활용률이나 latency를 주장하지 않습니다.

Softmax는 행 전체의 max와 sum을 사용하는 계산입니다

행을 나누어 처리하는 것과 한 행의 reduction 축을 나누는 것은 다릅니다.

동작을 따라 읽는 그림

계산 단계	설명용 값	연산 / 정밀도
입력	[2, 1, 0]	FP16
Max와 빼기	max=2 → [0, -1, -2]	행별 reduction + broadcast
Exp	[1, 0.3679, 0.1353]	FP16 vector 결과
Sum	약 1.5032	FP32 합산 → FP16 저장
Normalize	[0.6652, 0.2447, 0.0900]	각 exp / 합계

01 Max를 빼서 exp를 안정화

[2,1,0]에서 max=2를 빼 [0,-1,-2]를 만듭니다. Exp 결과는 [1,0.3679,0.1353]으로 큰 양수의 지수 overflow를 피합니다.

02 FP32 누산과 FP16 저장을 구분

Exp의 합은 내부에서 FP32로 누산하지만 저장된 합계와 다른 중간값은 FP16입니다. Softmax 전체를 FP32로 수행한다는 뜻이 아닙니다.

03 지원 길이는 수치 계약에 포함

유한 logits에서 exp(x-max)≤1이므로 마지막 축 길이 1..65504를 허용합니다. 더 긴 행을 자동 online reduction으로 바꾸는 경로는 현재 없습니다.

예제와 해석

약 1.5032로 나누면 [0.6652,0.2447,0.0900]입니다. 그림의 십진수는 수식 설명용이며 bit-exact FP16 dump가 아닙니다.

RMSNorm은 제곱 전에 입력을 줄여 overflow를 피합니다

원래 수식 y = x / sqrt(mean(x²)+eps) × weight에서 FP16 중간값의 범위를 고려합니다.

동작을 따라 읽는 그림

문제: 1000² = 1,000,000 → FP16 저장 범위를 초과

제곱을 FP16으로 만든 뒤 FP32로 더하는 것만으로는 해결되지 않음

1. 크기 축소

a = max(absmax(x), √eps)
u = x / a → |u| ≤ 1

→

2. 안전한 제곱·합

z = u / √width
s = sum(z × z)

→

3. 보정하여 출력

inv = 1/√(s+(√eps/a)²)
y = u × inv × weight

수식은 실수에서 등가, 실제 FP16 계산은 target 근사

LegalizeOps에서 정의한 수치 정책이며 자동 정밀도 탐색은 아님

01 FP32 합산만으로는 부족하다

x=1000이면 x²=1000000이므로 FP16 저장 범위를 넘습니다. 이미 overflow한 제곱값을 나중에 FP32로 더해도 원래 값을 되살릴 수 없습니다.

02 Legalize 단계에서 수식을 정의

a=max(absmax(x),sqrt(eps))로 나누고 z=(x/a)/sqrt(width)를 제곱·합산합니다. 줄어든 epsilon도 반영하여 원래 수식의 scale을 복원합니다.

03 수학적 등가와 FP16 결과는 다르다

실수 수학에서는 같은 식이지만 중간 FP16 반올림 때문에 target 근사입니다. Schedule이 임의로 정밀도를 개선하는 변환이 아니며 모델 기준과 별도로 검증합니다.

예제와 해석

u=x/a, s=sum((u/sqrt(width))²), y=u / sqrt(s+(sqrt(eps)/a)²) × weight. 합산 내부는 FP32입니다.

RoPE는 Q/K를 회전하고 GQA는 KV head를 공유합니다

위치 정보와 head 공유는 계산 의미이고, 복사·view·broadcast 여부는 후속 lowering의 결정입니다.

동작을 따라 읽는 그림

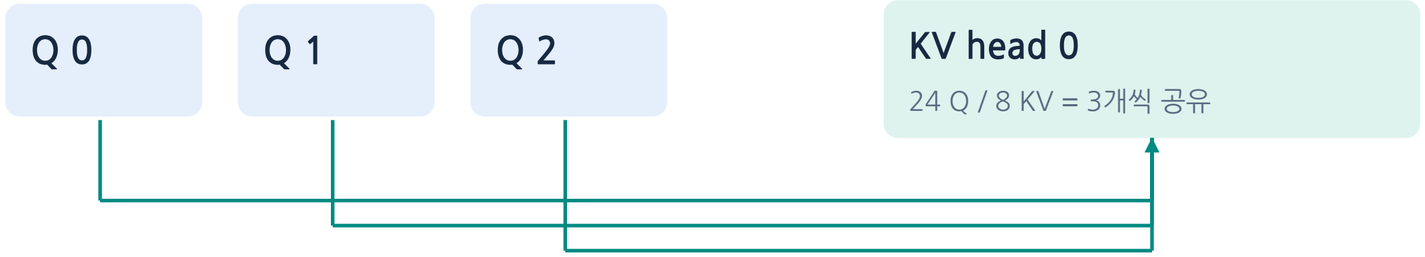
RoPE: head의 앞/뒤 절반

a = Q 또는 K의 앞 64개
b = 뒤 64개
V는 회전하지 않음

같은 position의 cos / sin

a′ = a × cos - b × sin
b′ = b × cos + a × sin
Host가 위치별 cos/sin 준비

GQA: KV head 하나를 Q heads 세 개가 공유



01 같은 head의 두 절반을 짝짓는다
Head dimension 128을 a와 b의 64개 원소씩 나눕니다. 새 a=a·cos-b·sin, 새 b=b·cos+a·sin으로 회전하고 V에는 적용하지 않습니다.

02 Position별 cos/sin은 host 준비
모델의 RoPE 설정과 현재 position을 사용합니다. 지원하는 default/linear/llama3 설정을 확인하며 알 수 없는 필드를 조용히 근사하지 않습니다.

03 24 Q heads가 8 KV heads 공유
KV head 하나를 세 Q heads가 사용합니다. Persistent cache 자체를 Q head 수만큼 복제할 필요는 없으며 repeat→broadcast matmul 변환도 후보로 검토합니다.

예제와 해석

Reshape가 view인 경우가 있어도 transpose·concat·repeat가 모두 복사 없는 연산인 것은 아닙니다. 실제 IR와 선택 결과를 확인합니다.

현재 attention fusion은 query 행을 나눕니다

QK.T → scale / mask → softmax → PV의 중간 score를 SRAM에서 소비합니다.

동작을 따라 읽는 그림

Score [S,S] · head 하나

열 방향: 모든 key 길이 S

선택한 query 64행만 SRAM에 생성

Q 조각 × K^T → score 조각

S=3072: 64×3072×2 = 384 KiB

↓

Scale / mask / softmax

한 query 행의 key 전체를 처리

↓

P 조각 × V → 출력 조각

다음 query 묶음으로 이동

Key online softmax는 현재 없음

01 전체 score를 만들면 매우 크다

S=3072, 24 heads에서 FP16 score 전체는 432 MiB입니다. Full-attention 후보는 query 일부 행의 score를 만들어 softmax와 PV에서 바로 소비합니다.

02 한 행의 key 방향은 유지한다

Head 하나의 query 64행 score는 64×3072×2=384 KiB입니다. 이외 K/V·mask·확률·출력·scratch도 SRAM에 필요합니다.

03 FlashAttention과 다른 범위

현재는 query 묶음을 줄여 공간에 맞춥니다. Key를 나누면서 max/sum/output을 보정하는 online softmax는 없고, decode query=1은 full-pattern 조건을 그대로 만족하지 않습니다.

예제와 해석

384 KiB는 score 조각 하나의 크기입니다. 전체 kernel의 SRAM 요구량이나 모든 shape에서 full fusion이 선택된다는 뜻이 아닙니다.

세 가지 메모리 계획이 서로 다른 범위를 봅니다

TVM의 lifetime 분석과 NPU의 커널 사이 SRAM 보관을 구분합니다.

동작을 따라 읽는 그림

범위	분석과 출력	어떤 문제를 푸는가
Relax 전체 함수	StaticPlanBlockMemory → storage pool	수명이 겹치지 않는 tensor의 기본 DRAM 공간 재사용
Scheduled TIR 한 kernel 안	USMP: allocation 충돌 → byte offset 배치	임시 buffer끼리 공간 공유 장치 상태는 보수적으로 보호
Kernel 사이 호출 순서	NPU memory search → 보관 / DRAM 선택	중간값을 마지막 소비까지 보관 남은 용량으로 schedule 재선택

서로 다른 범위의 세 결정이 합쳐져 최종 주소가 됨

01 Relax에서 기본 DRAM 재사용

CallTIRRewrite로 allocation과 호출을 드러낸 뒤 StaticPlanBlockMemory가 수명이 겹치지 않는 tensor에 storage pool을 공유시킵니다.

02 TIR에서 kernel 내부 SRAM 배치

USMP는 임시 buffer가 동시에 사용되는지 분석합니다. Backend는 그 결과와 정렬·상수·장치 상태의 보호 조건을 반영해 byte offset을 정합니다.

03 호출 사이에서는 보관을 선택

NPU memory search가 중간 tensor를 마지막 소비까지 SRAM에 남길지 비교합니다. 보관 공간을 예약한 나머지 용량으로 각 kernel을 schedule합니다.

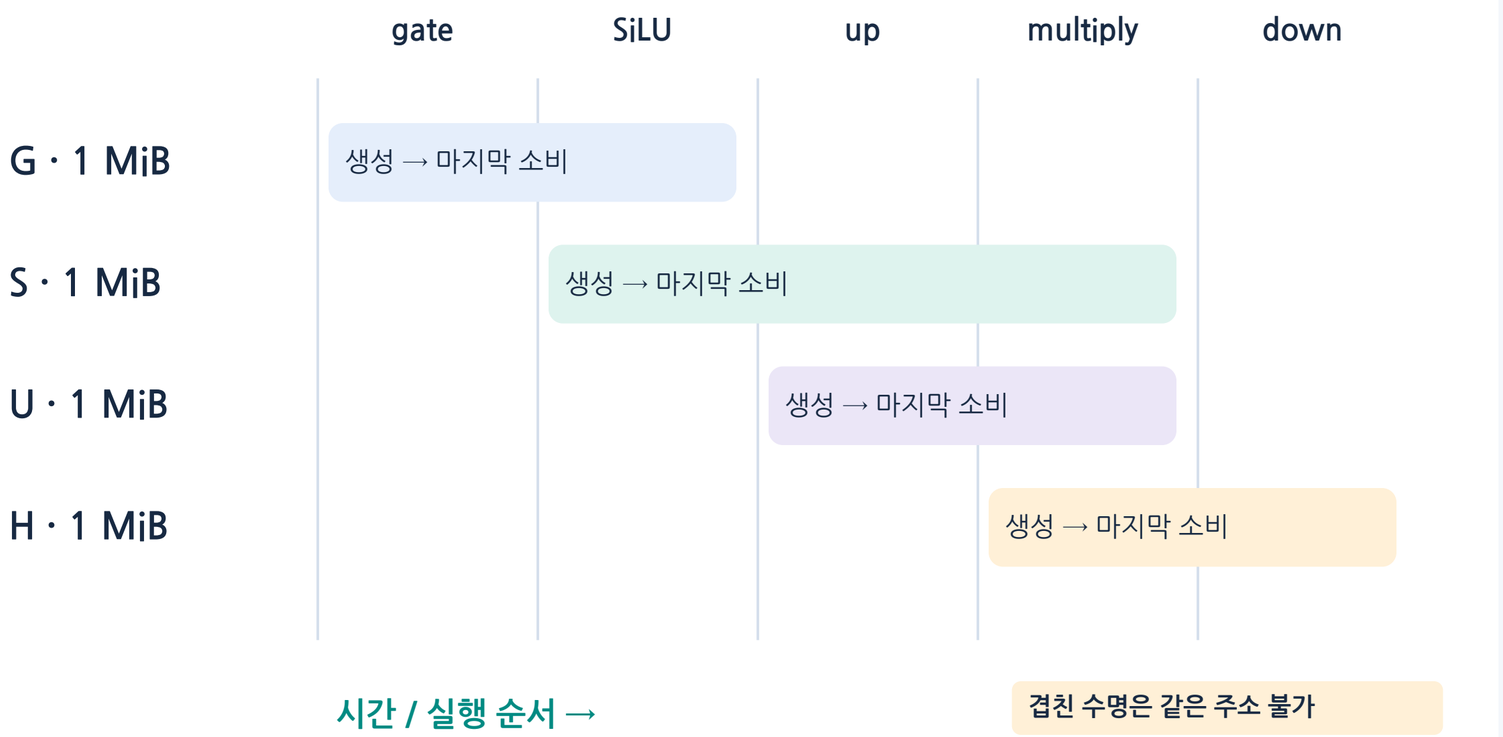
예제와 해석

USMP는 정적 allocation 분석·배치 도구입니다. Runtime GC나 persistent KV 관리, 전체 그래프의 SRAM 보관 정책을 자동으로 만들지는 않습니다.

값이 마지막으로 쓰인 뒤에 공간을 재사용합니다

Llama MLP의 gate → SiLU → up → multiply → down 순서를 봅니다.

동작을 따라 읽는 그림



01 수명의 끝은 마지막 읽기

G는 SiLU가 끝나면 필요 없지만 S는 up을 계산하는 동안에도 살아 있어야 합니다. H는 down이 소비할 때까지 유지합니다.

02 공간 재사용은 값의 변환이 아니다

G가 죽은 공간에 U를 써도 되는 것은 G의 옛 비트가 더 이상 필요 없기 때문입니다. 크기와 정렬이 맞는지 확인하고 살아 있는 값끼리는 겹치지 않습니다.

03 단순 크기 합보다 peak가 커질 수 있다

Weight panel·임시값·정렬·빈 공간도 필요합니다. 현재 호출 순서를 바꾸거나 tensor 일부만 남기는 정책은 없으며 큰 값은 DRAM 경로를 선택할 수 있습니다.

예제와 해석

M=64 Llama MLP에서 G/S/U/H는 각각 1 MiB입니다. Residual도 마지막 add까지의 실제 수명과 작업 공간을 함께 평가합니다.

커널을 나누어도 H를 SRAM에 계속 둘 수 있습니다

44 KiB 예제: X[64,64] @ W1[64,128] → H → H @ W2[128,64]

동작을 따라 읽는 그림



01 두 kernel이 같은 H 주소를 공유

첫 matmul이 H[64,128]을 SRAM에 만들고, 두 번째가 그 주소를 입력으로 읽습니다. H를 위한 DRAM 저장과 재읽기가 모두 사라집니다.

02 작업 공간은 kernel마다 바뀐다

X가 죽은 공간은 두 번째 kernel의 Y 작업 공간으로, W1 공간은 W2로 재사용합니다. H 영역은 두 kernel 사이에서 계속 유효하게 유지합니다.

03 보관은 남은 용량으로 검증

44 KiB 예제의 실제 peak는 42508 bytes입니다. H 16 KiB 외에 상수·scratch·현재 입력과 weight가 함께 들어가는지 확인한 뒤 선택합니다.

예제와 해석

주소 단위는 bytes입니다: X=1548, W1=9740, H=26124. 상수와 scratch 때문에 tensor가 SRAM 주소 0에서 시작하지 않습니다.

최종 H에 직접 쓰고, 쓰지 않는 DRAM 예약을 없앱니다

별도 출력 tile을 거치는 복사와 불필요한 image 공간을 구분해 줄입니다.

동작을 따라 읽는 그림

H [64,128] FP16: 한 행 = 256 bytes

왼쪽 64열

시작: H

오른쪽 64열

시작: H + 128 B

완료된 accumulator를 직접 SAVE

부분 영역 view가 전체 H의
row stride 256 B를 유지

DRAM 예약 정리

초기: H용 DRAM pool 존재
선택: H를 SRAM-only로 전달
후처리: 완전히 미사용인 pool 제거

Tile 8 KiB를 단순 연속 배치한 주소와 다름

01 SAVE 대상을 H의 부분 영역으로

별도 64×64 출력 tile에 저장한 뒤 복사하는 대신 H의 왼쪽/오른쪽 64열에 직접 SAVE합니다. 전체 K 누산을 마친 값의 저장 위치만 바꿉니다.

02 열 tile은 행 stride를 유지

H[64,128] FP16의 한 행은 256 bytes입니다. 오른쪽 tile 시작은 H+128 bytes이며 H+8192가 아닙니다. Tile 전체가 연속한 덩어리라고 가정하면 틀립니다.

03 완전히 안 쓰는 DRAM pool 제거

SRAM-only 전달이 확정된 뒤 어디에서도 쓰지 않는 pool을 없앱니다. 남은 DMA·I/O·상수·cache 주소 기준도 함께 갱신합니다. 일부가 쓰이는 pool은 유지합니다.

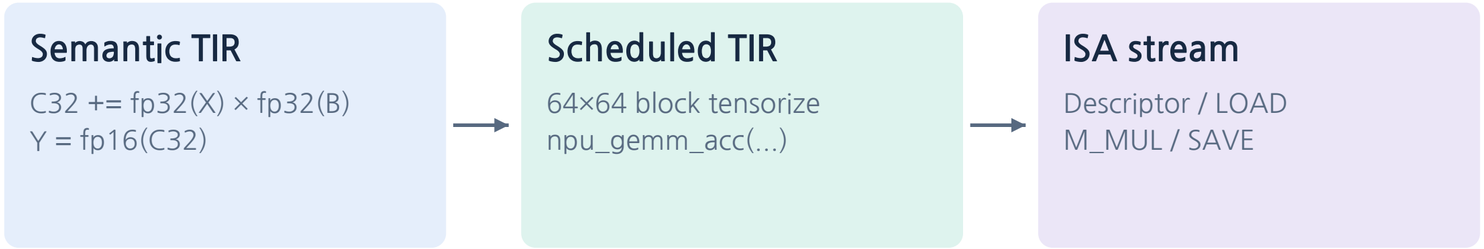
예제와 해석

보존된 두 matmul 비교: words 85→77, image 65548→49164 bytes. DMA와 SRAM peak는 같고 출력은 bit-exact입니다.

TIR의 계산과 복사를 실제 명령으로 펼칩니다

프로그램은 정적으로 전개된 명령 stream입니다. TIR loop가 ISA loop로 남지는 않습니다.

동작을 따라 읽는 그림



실제 word offset	명령의 의미	누산 관계
30 / 44	첫 matmul의 H 왼쪽 / 오른쪽 tile	둘 다 MAC=0: 다른 출력
60 / 67	두 번째 matmul의 K tile 두 개	MAC=0 다음 MAC=1
71 / 72 / 76	Y SAVE / GSTORE / HALT	K 완료 뒤 외부 출력 저장

01 Tensorize는 intrinsic 호출을 만든다

수식·dtype·scope가 맞는 64×64 block을 npu_gemm_acc 같은 호출로 바꿉니다. 이 호출 자체는 아직 최종 opcode나 program word가 아닙니다.

02 Codegen은 descriptor와 명령을 만든다

MAIN/PARTIAL descriptor에 shape·stride·접근 영역을 설정하고 LOAD, MAC bit를 가진 M_MUL, SAVE로 펼칩니다. 복사 block은 DMA나 SRAM 이동이 됩니다.

03 Linker는 실제 상태를 보며 정리

Kernel을 호출 순서로 연결하고 주소의 읽기·쓰기 효과를 추적합니다. 같은 명령이 반복되어도 중간 값이 덮어써졌으면 무조건 지울 수 없습니다.

예제와 해석

77 words 예제는 62개 명령입니다. DMA 한 개가 4 words이므로 저장 word 수와 실행 instruction 수가 다릅니다.

같은 주소 숫자도 필드마다 단위가 다릅니다

실제 두 matmul의 X GLOAD packet을 byte 주소로 읽어 봅니다.

동작을 따라 읽는 그림

X GLOAD · word offset 8 · 4-word packet

000c18a0	00000000	00000800	00010800

필드 / 위치	원래 값과 단위	Byte로 읽기
SRAM 주소	0xC18 = 3096 nibbles	3096 ÷ 2 = 1548 B
DRAM 시작	0 cells	0 × 4 = 0 B
전송량	0x800 = 2048 cells	2048 × 4 = 8192 B
다른 필드 주의	Plan은 기본 2 B / stride는 원소	FP16 stride 128 = 256 B

01 SRAM 필드는 nibble 단위

첫 word의 SRAM 필드는 0xC18=3096입니다. Nibble은 반 byte이므로 byte 주소는 3096÷2=1548로, 실제 X의 배치와 일치합니다.

02 DMA global 쪽은 32-bit cell

이 packet의 global 시작은 0 cell, 전송량은 0x800=2048 cells입니다. 2048×4=8192 bytes를 DRAM에서 SRAM으로 가져옵니다.

03 Plan과 shape도 따로 해석

기본 plan.address는 2-byte unit, shape/row stride는 element 단위입니다. FP16 stride 128은 256 bytes이며 단위를 섞으면 2배·4배 주소 오류가 납니다.

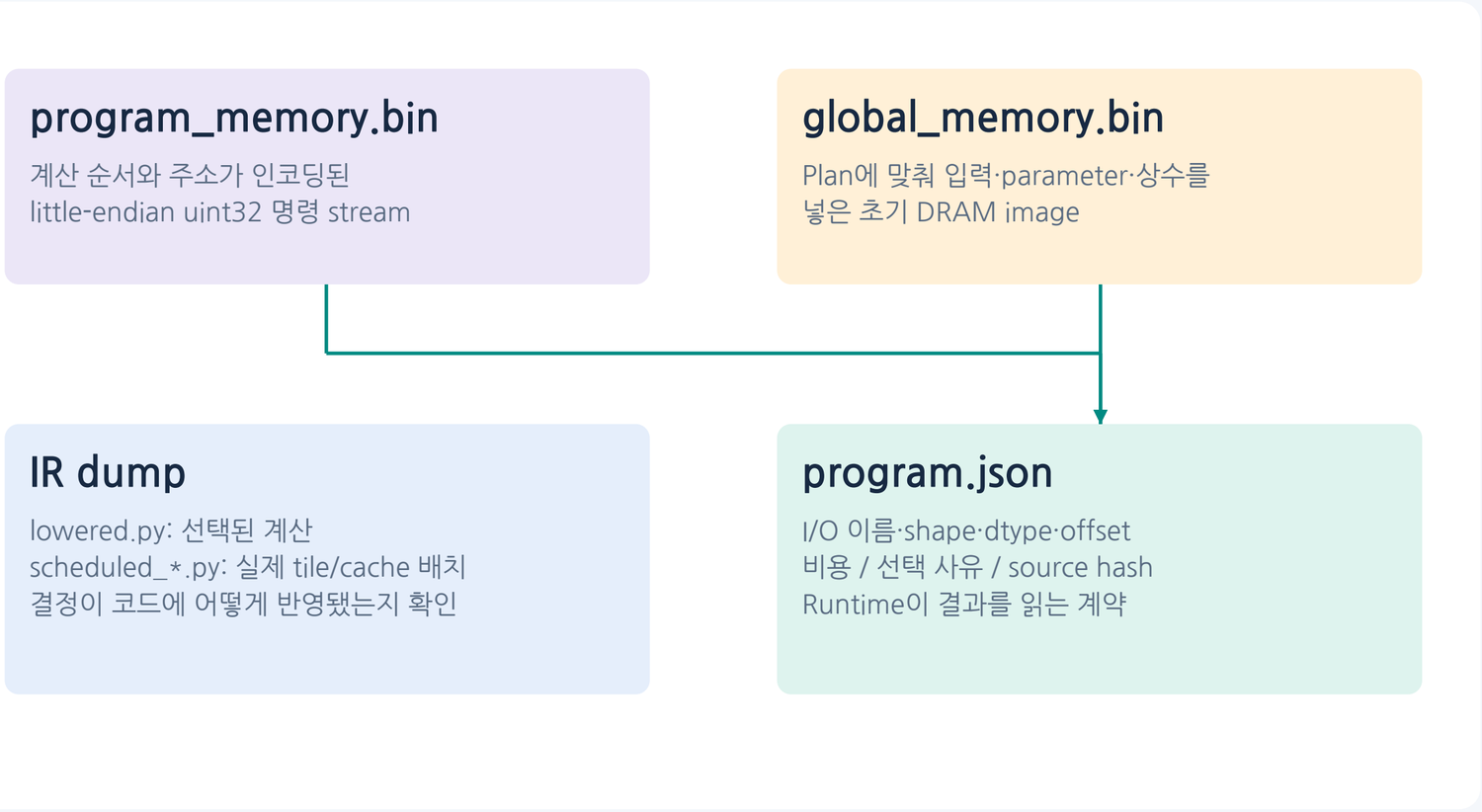
예제와 해석

Packet @ word offset 8: 000c18a0 00000000 00000800 00010800. 이 예의 opcode A0는 GLOAD입니다.

명령과 메모리 계약을 함께 내보냅니다

NpuExecutable = words + plan + 선택된 IR + 입력 signature

동작을 따라 읽는 그림



01 명령과 값은 별도 파일

program_memory.bin은 명령 stream이고 global_memory.bin은 입력·weight를 넣은 초기 DRAM image입니다. Binary만으로 결과를 재현할 수는 없습니다.

02 Plan은 서로의 연결 계약

program.json은 주소·통계·선택 진단과 출력 tree의 shape/dtype/offset을 담습니다. Runtime은 이 정보를 따라 값을 넣고 결과를 읽습니다.

03 IR dump로 선택 이유를 추적

lowered.py는 선택된 계산, scheduled 파일은 실제 호출의 배치를 설명합니다. 작은 fused 예제는 실제 cache recipe에서 dump하여 선택 결과와 연결했습니다.

예제와 해석

재현에 필요한 묶음: 같은 명령 + 같은 plan + signature에 맞는 입력/weight + 같은 실행 계약. Words는 cycle 추정값이 아닙니다.

Prefill 이후에는 새 token 하나씩 처리합니다

Host는 embedding·위치·mask를 준비하고 NPU가 28-layer 계산을 수행합니다.

동작을 따라 읽는 그림

호출	NPU에 넣은 token / position	Cache 유효 길이	예측 token
Prefill	Prompt 7 tokens	7	358 · “ I”
Decode 1	358 / position 7	8	2846 · “'m”
Decode 2	2846 / position 8	9	4560 · “ trying”

Host 준비 → NPU 28 layers → host argmax → 다음 입력

마지막 예측 token 4560은 아직 입력으로 처리되지 않았으므로 그 K/V도 아직 없음

01 Prefill이 첫 token을 예측

Host가 prompt의 embedding·cos/sin·mask를 준비하면 NPU가 28 layers와 LM head를 계산합니다. Logits의 greedy argmax는 host가 고릅니다.

02 Decode 입력은 방금 예측한 token

첫 예측 358을 position 7에 넣어 다음 token 2846을 얻습니다. 다시 2846을 position 8에 넣어 4560을 얻습니다.

03 예측과 cache 반영 시점이 다르다

마지막 4560은 아직 NPU 입력으로 처리하지 않았으므로 그 token의 K/V는 없습니다. 생성 3개에 필요한 호출은 prefill 1회와 decode 2회입니다.

예제와 해석

보존된 prompt “Hello, NPU compiler!”는 7 tokens이며, 생성 문자열은 “ I'm trying”입니다.

Cache capacity와 현재 유효 길이는 다릅니다

예: capacity C=9, 기존 유효 token 수 L=7, 현재 decode token 1개

동작을 따라 읽는 그림

이번 attention의 입력: capacity 9 + 현재 token 1



Persistent cache에 기록: 첫 빈 슬롯 L=7

Attention에서 현재 token의 임시 위치는 C=9, 보존할 최종 위치는 L=7

Mask는 7,8번의 기여를 막음 / 고정 shape의 연산량이 자동으로 줄지는 않음

01 C는 고정 shape, L은 현재 길이

Decode attention은 기존 C개 cache 슬롯과 현재 token 하나를 논리적으로 연결한 C+1 길이입니다. Step마다 L만 늘어도 계산 shape는 같습니다.

02 빈 슬롯은 mask로 기여를 막는다

C=9, L=7이면 0..6과 현재 위치 9는 유효하고 7,8은 제외합니다. Mask는 기여를 막지만 해당 위치의 모든 계산을 동적으로 생략하지는 않습니다.

03 저장할 위치는 첫 빈 슬롯 L

현재 token은 이번 attention의 임시 위치 C에 있지만 persistent cache에는 L=7에 기록합니다. 다음 step은 L=8에 쓰며 같은 base 프로그램을 재사용합니다.

예제와 해석

C를 늘리면 재컴파일 없이 처리할 길이가 늘지만 빈 슬롯 계산도 늘 수 있습니다. 여러 capacity bucket은 이 교환 관계를 조절합니다.

새 K/V의 실제 이동은 NPU DMA가 수행합니다

기본 decode session은 별도 DRAM 출력 뒤에 append 명령을 붙입니다.

동작을 따라 읽는 그림

새 K/V 계산 결과
별도 DRAM 출력
K/V 합계 114688 B

→

NPU DMA append
Host가 position 주소 patch
기록된 DMA 458752 B

→

Persistent cache
Layer·head별 슬롯 L
다음 decode 호출에 사용

층	기록된 동작	섞어 세면 안 되는 이유
Host patch	작은 입력 6676 B / suffix 갱신	Weight rewrite 절감과 관련
NPU DMA	위치·layout에 따른 cache 이동	실제 명령의 DRAM 전송
Simulator I/O	매 호출 전체 image 파일 read/write	C-model transport 비용

01 새 K/V는 별도 출력으로 생긴다

기본 DecodeSession에서 NPU 계산은 새 K/V를 별도 DRAM output에 씁니다. 처음부터 persistent cache의 최종 슬롯에 직접 쓰는 경로는 아직 없습니다.

02 Host가 주소를 정하고 NPU가 이동

Host는 position에 맞춰 append tail의 주소를 patch합니다. 실제 데이터 전송은 program 끝에 붙인 NPU DMA가 수행합니다. 매 step CPU 복사로만 처리하는 방식은 아닙니다.

03 전송량의 층을 분리해서 본다

기록의 입력 patch는 6676 B, 새 K/V는 114688 B, append DMA는 458752 B입니다. C-model은 호출마다 전체 image 파일을 읽고 쓰므로 그 I/O도 별도입니다.

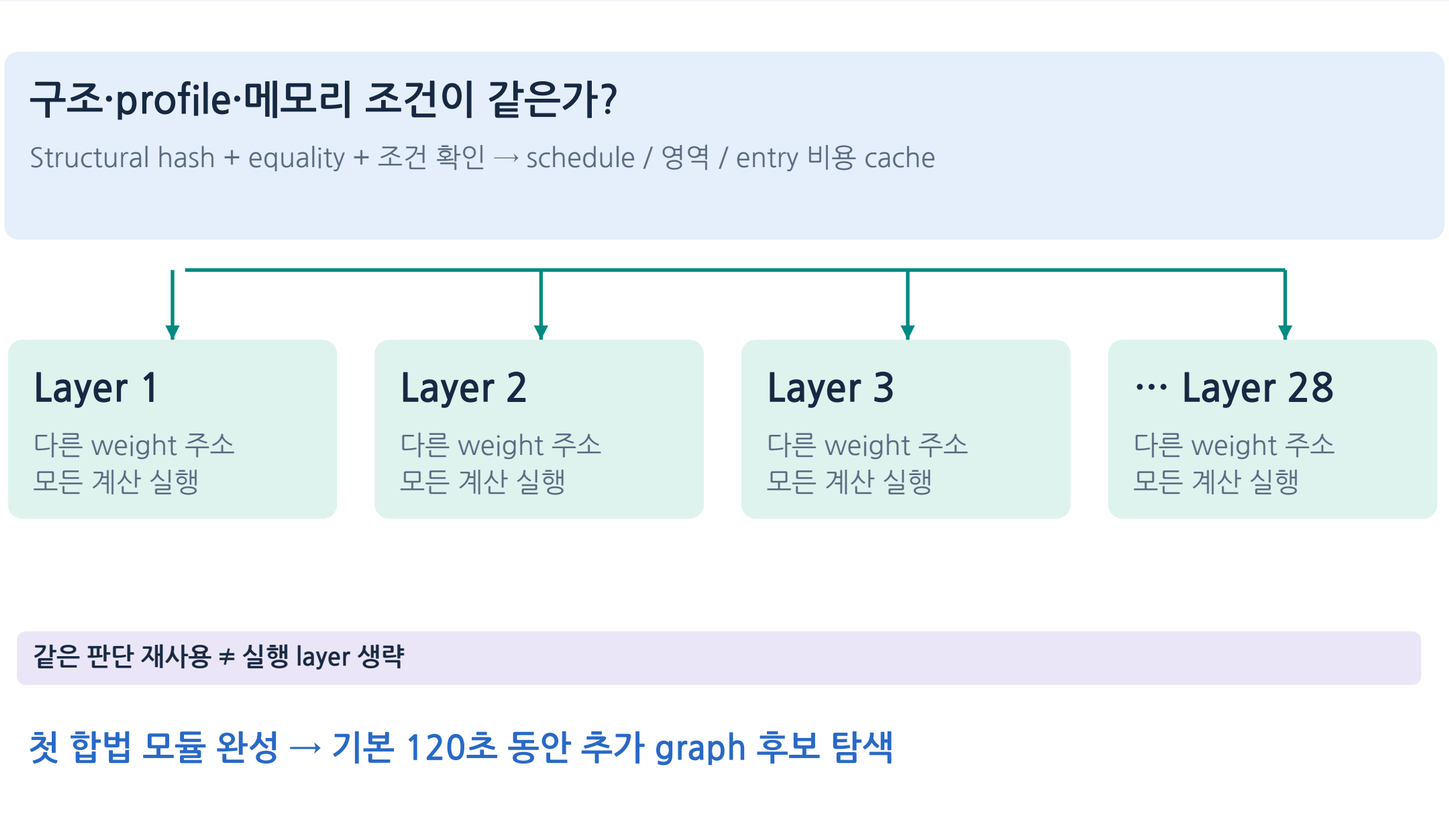
예제와 해석

Prefill→capacity cache 초기화는 host 복사입니다. 이후 decode의 persistent DRAM 상태와 호출 간 SRAM 유지 여부는 다른 계약입니다.

반복 layer의 판단을 재사용해 compile 작업을 줄입니다

실행에는 28개 layer가 모두 포함됩니다. 같은 판단을 반복하지 않도록 합니다.

동작을 따라 읽는 그림



01 같은 구조의 결정은 다시 쓰기

IR 구조와 profile·메모리 조건이 같을 때 schedule·영역·entry 비용을 cache합니다. Shape만 같다고 서로 다른 계산을 하나로 취급하지 않습니다.

02 주소와 실행은 layer마다 필요

28개 layer의 weight와 값 수명은 다릅니다. 최종 entry stream에는 모든 layer의 필요한 계산이 포함되며 compile 판단을 재사용해 실행을 생략하는 것은 아닙니다.

03 120초는 추가 탐색의 예산

120초는 첫 합법 모듈 이후의 기본 추가 탐색 시간입니다. None이면 해제됩니다. Schedule 후보 수는 별도 한도이며 이번 256-token 실험은 16개 성공 목표·128회 시도입니다.

예제와 해석

실패 후보의 중복 검사를 피하고 시도 한도를 둡니다. 작업 경계에서 시간을 확인하므로 긴 TVM 작업 하나만큼 예산을 넘길 수 있습니다.

수치 결과와 비용을 서로 다른 근거로 확인합니다

모든 숫자를 하나의 성능 점수로 합치지 않습니다.

동작을 따라 읽는 그림

검증 / 지표	확인하는 것	해석의 한계
Bit equality / cosine	수치 의미 / 모델 결과	짧은 입력이 모든 품질을 대표하지 않음
DMA bytes	명령이 요청한 DRAM 이동	Host patch / 파일 I/O와 다름
ISA words	명령 저장 공간	Hardware latency와 다름
SRAM peak / image	최고 SRAM / DRAM 예약 크기	전송량과 다름
Compile seconds	Compiler의 wall-clock 비용	Inference 속도와 다름

01 정확성: 무엇과 비교했는가

작은 kernel은 bit equality·LLVM 기준을, 전체 Llama는 HF argmax·logits cosine을 확인합니다. 같은 출력의 근거와 단순 compile 성공을 구분합니다.

02 비용: 어떤 자원을 세었는가

DMA는 전송 bytes, words는 명령 저장 크기, SRAM peak는 최고 필요 공간, image는 DRAM 예약 크기입니다. 서로 같거나 비례한다고 가정하지 않습니다.

03 근거: 언제 무엇을 실행했는가

문서의 작은 예제 실행, 보존된 전체 모델 실행, 정적 MLP compile, 과거 단일 시간 관측을 표시합니다. 이번 발표 업데이트로 새 compiler benchmark를 만들지 않았습니다.

예제와 해석

실장치 latency·긴 문맥 품질·양자화 generation은 이 자료의 검증 결과가 아닙니다. 각 실험의 조건을 숫자와 함께 읽습니다.

Fusion은 DMA가 같아도 명령을 줄일 수 있습니다

Linear + bias + SiLU · [64,128] → [64,64] · 두 구성 모두 SRAM 전달 사용

동작을 따라 읽는 그림

ISA words · 명령 저장 크기



지표	Off	Auto	결과
DMA bytes	41100	41100	같음
SRAM peak	50700	42508	8192 B 감소
출력	FP16	FP16	Bit-exact / LLVM max diff 0

같은 전송량에서도 명령과 작업 공간은 줄어들 수 있음

01 동일한 입력으로 두 구성 실행

X[64,128]·W[64,128]·bias[64], seed 914를 사용한 작은 예제입니다. Off 3 kernels와 auto 1 kernel 모두 SRAM 전달을 사용합니다.

02 DMA보다 명령과 공간이 줄었다

전송량은 둘 다 41100 B입니다. Words는 2812→1577, SRAM peak는 50700→42508 B로 줄었습니다. Fusion의 효과가 항상 DRAM 감소로만 나타나지는 않습니다.

03 출력과 counter도 확인

두 출력은 FP16 bit-exact이고 이 입력의 LLVM 기준 최대 절대차는 0입니다. C-model의 DMA counter도 정적 명령 분석과 일치했습니다.

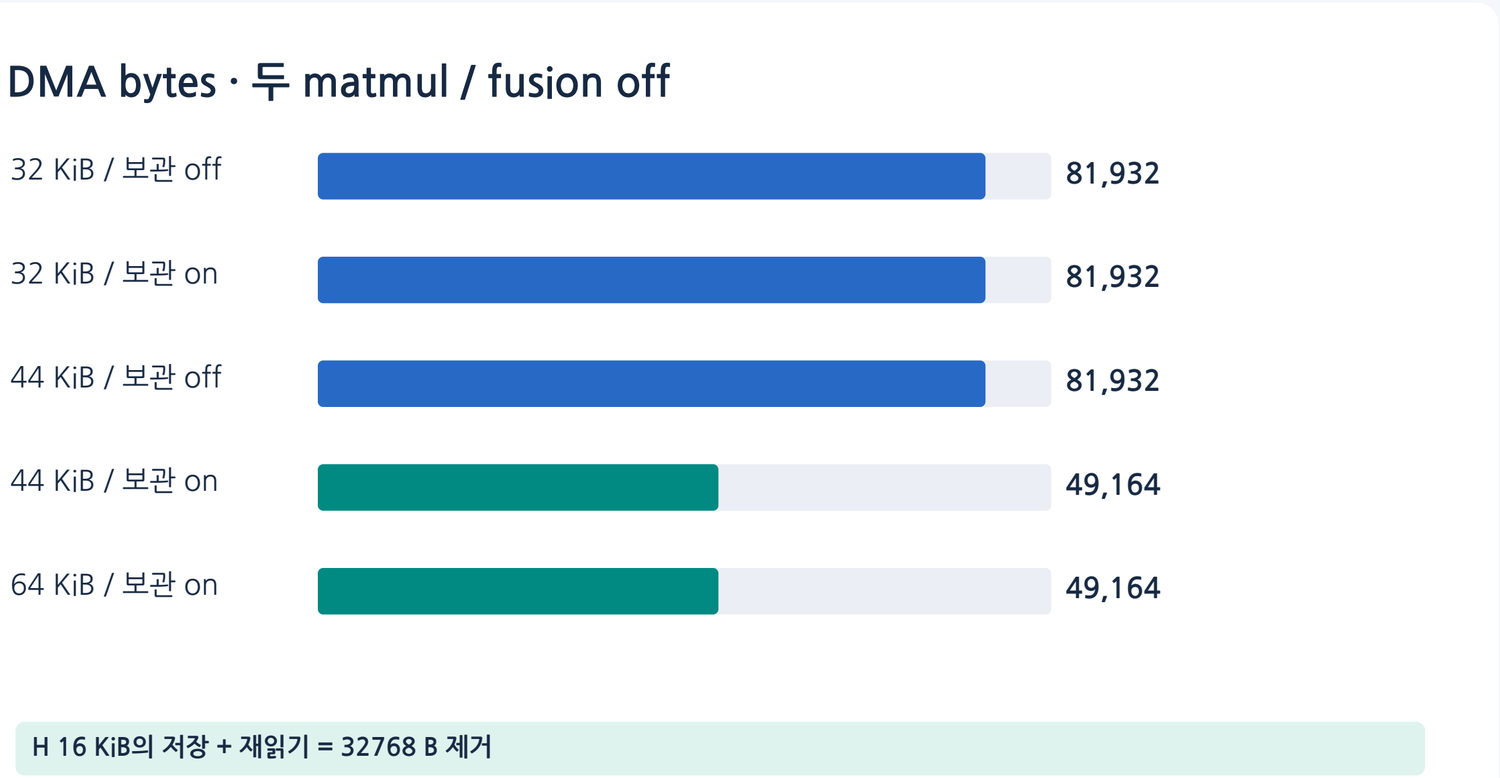
예제와 해석

한 작은 예제의 결과입니다. 모든 fusion이 유리하거나 words 감소율만큼 실제 장치가 빨라진다고 확대하지 않습니다.

44 KiB부터 H의 DRAM 왕복이 사라졌습니다

두 matmul · fusion off · SRAM 보관 탐색과 용량만 변경 · 출력은 모두 동일

동작을 따라 읽는 그림



01 용량과 보관 옵션만 바꾼다

X[64,64]→H[64,128]→Y[64,64]인 두 matmul, fusion off 조건입니다. 32/44/64 KiB와 whole-tensor 보관 탐색을 비교했습니다.

02 44 KiB에서 보관이 성립한다

H 16 KiB를 저장하고 다시 읽는 32768 B가 사라져 DMA가 81932→49164 B로 줄었습니다. 32 KiB는 옵션을 켜도 같은 결과였고 64 KiB는 추가 이득이 없었습니다.

03 비교에서 유지된 기능도 있다

Joint-memory off여도 기존 residency·peephole은 사용합니다. 모든 메모리 최적화의 on/off 비교가 아닙니다. 다섯 구성의 출력과 DMA counters를 검증했습니다.

예제와 해석

용량을 늘리는 자체보다 살아 있는 H와 작업 공간이 함께 들어가는지가 중요합니다. 이 예제의 임계점을 전체 모델에 일반화하지 않습니다.

DRAM 예약 제거는 전송량과 별개로 image를 줄입니다

개선 전후의 조건과 검증 수준을 구분합니다.

동작을 따라 읽는 그림

기록 / 범위	Words 전 → 후	DRAM image 전 → 후
E3 · 두 matmul 실행 / bit-exact	85 → 77	65548 → 49164 B 16 KiB 예약 제거
E4 · 실제 폭 MLP 정적 compile	143404 → 140594	154927116 → 151781388 B 3 MiB 예약 제거

두 비교 모두 DMA와 SRAM peak는 그대로

이미 SRAM으로 전달하던 중간값의 불필요한 명령·DRAM 예약을 정리한 결과

01 E3: 작은 예제의 실행 결과

이미 SRAM 보관을 사용하는 상태에서 pointwise 정리·직접 출력·pool 제거를 비교했습니다. Words 85→77, DRAM image 16 KiB 감소, 출력은 bit-exact입니다.

02 E4: 실제 폭 MLP의 정적 결과

M=64·hidden 3072·MLP 8192에서 5 kernels를 유지합니다. Words 143404→140594, image 3 MiB 감소이며 이 MLP 측정은 executed=false입니다.

03 공간과 전송은 별개의 지표

이미 SRAM에서 전달되던 값의 미사용 DRAM 예약을 지워도 DMA는 더 줄지 않을 수 있습니다. 같은 pool을 여러 중간값이 재사용하므로 제거량은 tensor 크기의 단순 합도 아닙니다.

예제와 해석

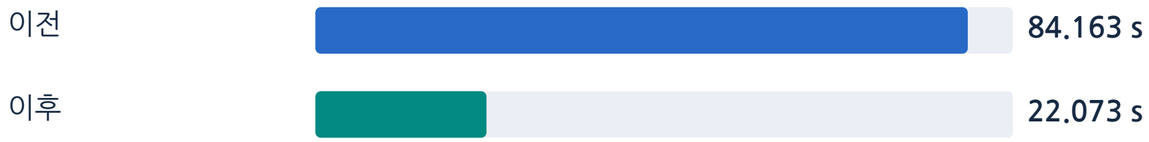
E2는 보관 옵션 비교(87→77), E3는 보관 이후 개선 비교(85→77)입니다. 서로 다른 기준을 하나의 전후 수치로 섞지 않습니다.

같은 프로그램을 얻는 compile 작업이 줄었습니다

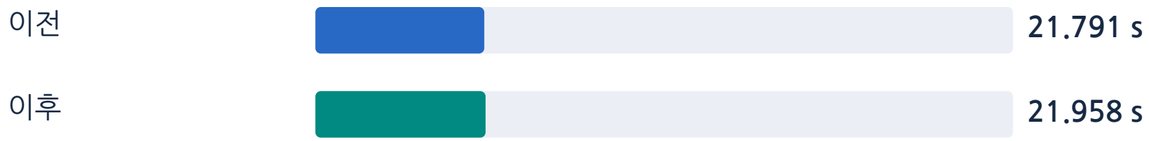
실제 폭 Llama MLP · 동일 binary 확인 · 반복 평균이 아닌 wall-clock 관측

동작을 따라 읽는 그림

M=7 · 실패 검사와 중복 시도 감소



M=64 · 두 경우 모두 47 trials



두 비교 모두 최종 binary hash 동일 / 과거 단일 관측

01 M=7에서는 불필요한 검사가 감소

기록된 compile 시간은 84.163→22.073초이고 TIR trial은 347→112회입니다. 최종 143883-word binary의 hash는 동일합니다.

02 M=64에서는 제한이 걸리지 않음

무제한과 64회 제한이 모두 47 trial을 사용했습니다. 시간은 21.791→21.958초로 비슷하고 140594-word binary도 같습니다.

03 반복 benchmark로 읽지 않는다

기존 단일 wall-clock 관측이며 일부는 다른 검증과 동시에 실행됐습니다. 모든 shape에서 일정 배율 개선이나 전체 모델 cold compile 시간을 보여주는 표가 아닙니다.

예제와 해석

개선의 목적은 같은 합법 프로그램을 얻기 위해 반복하던 판단을 줄이는 것입니다. 탐색 한도가 실제로 작동했는지도 함께 봅니다.

과거 7-token 전체 모델에서 HF와 같은 token을 생성했습니다

E6 보존 기록입니다. 새 256-token prefill의 검증 결과와 구분합니다.

동작을 따라 읽는 그림

실행	HF와 같은 token	Cosine	최대 절대차
Prefill	358	0.9999977148	0.03515625
Decode pos 7	2846	0.9999958473	0.02343750
Decode pos 8	4560	0.9999929546	0.03906250

생성: “ I'm trying”

유한 logits + token 일치 + cosine ≥ 0.999 확인

01 28 layers를 실제 실행한 기록

Llama 3.2 3B FP16, prompt 7 tokens, capacity 9, greedy 3 tokens 조건입니다. Prefill과 position 7/8 decode 모두 HF argmax와 일치했습니다.

02 통과 기준과 기록값을 구분

유한한 logits, token 일치, cosine≥0.999가 기준입니다. 최대 절대차는 기록했지만 별도의 통과 threshold로 사용하지 않았습니다.

03 이 결과의 검증 범위

짧은 한 prompt의 생성 기록입니다. 장문·다양한 prompt·양자화 품질은 별도이며 전체 graph 탐색도 예산 종료로 unfused 기준을 선택했습니다.

예제와 해석

이번 발표 갱신에서 전체 모델을 다시 컴파일하지 않았습니다. 보존된 실행 로그·JSON·source hash로 결과를 추적합니다.

탐색 순서와 결과 재사용에 TVM 기능을 연결했습니다

후보 생성, 검사 순서, 실제 비용 측정은 서로 다른 역할입니다.

동작을 따라 읽는 그림

후보를 만드는 규칙
Panel · K chunk · loop 순서
cache 위치를 조합

→

검사 순서 선택
기본 space / 선택형 ReplayFunc
동일한 NPU 후보 공간 사용

→

실제 비용 확인
SRAM 배치 + 명령 생성
DMA → words → SRAM

TVM 기능	현재 연결한 역할
Workload / JSONDatabase	계산 구조와 선택된 schedule trace 보관
reuse / tune	기록 재생·검증 / 기존 winner와 새 후보 비교
ReplayFunc	후보 선택을 반복; 학습형 비용 모델은 아님

같은 계산도 SRAM 용량·입출력 배치가 다르면 다시 확인

01 중복과 확실한 실패 줄이기
같은 순회·복사 주기가 되는 후보를 정규화합니다.
직접 FP16 입력의 cache만으로 SRAM을 초과한다는 증명이 가능한 경우에만 후보를 미리 제외합니다.

02 검사는 실제 NPU 명령으로
후보마다 TVM Schedule API, SRAM 배치, 명령 생성을 수행합니다. Words 예산은 기준 프로그램 크기를 반영하며 작은 설정 하나로 큰 커널의 탐색을 바로 막지 않습니다.

03 TVM 기록은 선택해서 사용
reuse는 trace 재생과 검증, tune은 추가 탐색입니다.
장치·수치·메모리 조건이 맞아야 재사용합니다.
ReplayFunc는 현재 후보 풀의 검사 순서를 바꾸는 선택지입니다.

예제와 해석

같은 shape만으로 재사용하지 않습니다. 계산 구조와 SRAM·입출력 조건을 확인하고, layer별 최종 주소는 다시 연결합니다.

512-token down에서는 탐색을 늘려도 같은 명령이 선택됐습니다

E9는 실제 Llama weight를 쓰는 한 projection의 비교입니다.

동작을 따라 읽는 그림

512-token down / E9	Space	ReplayFunc
추가 합법 후보	4개	4개
커널 탐색 시간	10.586 s	33.264 s
최종 DMA bytes	112,197,644	112,197,644
최종 ISA words	372,637	372,637
최종 SRAM peak	5,349,388 B	5,349,388 B

동일 binary · C-model 출력 검사 통과

실제 layer-0 weight + 합성 activation / 전체 Llama 출력 검증은 아님

01 탐색이 실제로 진행된 결과

Space와 ReplayFunc 모두 추가 합법 후보 4개를 평가했습니다. 그 범위에서는 기존 schedule보다 정적 비용이 좋은 후보를 얻지 못했습니다.

02 개선된 것은 compile 작업

중복·확실한 용량 초과 검사와 재사용을 보완했습니다. 최종 명령이 같으므로 이 사례에서 실행 성능이 좋아졌다고 주장하지 않습니다.

03 전체 모델과 범위를 구분

실제 checkpoint weight, 합성 activation을 사용했습니다. 전체 28-layer의 256-token 실험은 다음 페이지의 별도 E10입니다.

예제와 해석

실험 시간은 단일 관측입니다. ReplayFunc가 더 좋은 결과를 보장하지 않으며, 측정한 후보 전체를 넘어 최적이라고 주장하지 않습니다.

256-token 전체 prefill은 9개 graph 구성을 비교합니다

E10 중단: 큰 cache fusion의 명령 생성에서 메모리 증가. 병목 수정 후 전체 재검증은 남았습니다.

동작을 따라 읽는 그림

256-token prefill	Generic off	Vectors	Matmul
Attention off	기준 구성	영역 10종	영역 11종
Attention score	1개 구성	영역 9종	영역 10종
Attention full	1개 구성	영역 9종	영역 10종

전체 graph 구성 9개

같은 IR의 GQA·producer 조합은 제거
영역 수는 종류이며 최종 채택 수가 아님

메모리 증가로 중단

기준 1개 완료 / 두 번째 구성에서 중단
병목 수정 · 전체 출력 재검증은 남음

28 layers 전체 · prefill만 · HF logits와 layer별 K/V 비교 예정

01 비교 조건을 동일하게

분리된 커널도 SRAM 보관과 직접 출력을 사용합니다. Fusion이 이보다 DMA·명령·SRAM 비용을 줄이는지 전체 prefill 기준으로 확인합니다.

02 반복 layer는 판단을 재사용

계산·메모리 조건이 같은 recipe는 재사용합니다. 서로 다른 보관 선택으로 가용 SRAM이 달라지면 추가 검사하고, 실제 주소와 전체 수명은 모델 전체에 반영합니다.

03 병목 수정과 전체 검증을 구분

생산자 반복 배치를 거절하고 행 분할의 출력 덮어쓰기 오류를 수정했습니다. 독립·회귀 검사는 통과했지만 전체 256-token 모델은 다시 실행하지 않았습니다.

예제와 해석

9개 중 기준 구성 1개 완료 후 중단했습니다. 병목 독립 검사 통과와 전체 모델 출력 검증을 구분합니다.

근거: 11_experiments.md, llama_prefill256_full_search_progress.json, codegen_bottleneck_0929.json

효과가 큰 항목부터 근거를 확인하며 확장합니다

현재 지원·검증 범위를 넘어서는 기능은 후속 검토로 남깁니다.

동작을 따라 읽는 그림

검토 수준	대표 항목	먼저 확보할 근거
장치 계약 확인	Vector 256 / 긴 VLEN Cycle와 실제 실행기	RTL 동작 / tail / buffer 대표 연산 timing
제한된 효과 비교	작은 fusion / KV 직접 저장 Capacity bucket	최종 words·DMA·space 구현과 ABI 변경 범위
별도 큰 설계	Online softmax / tile streaming Eviction / 정밀도 탐색	실제 손실이 큰 workload 장치 상태와 검증 기준

01 먼저 장치 계약을 확인할 부분

Vector 256과 긴 VLEN의 LOAD/연산/SAVE 동작을 RTL에서 확인해야 합니다. Cycle 자료가 없으므로 현재 DMA→words→SRAM 비용 모델의 한계도 분명합니다.

02 이득을 측정한 뒤 설계할 부분

작은 fusion 영역, KV 최종 슬롯 직접 저장, capacity bucket은 대표 입력에서 이득을 먼저 봅니다. 표준 API로 공통 처리할 수 있는지도 함께 검토합니다.

03 큰 변경은 별도 범위로 다룬다

Key online softmax, graph tile streaming, eviction·호출 순서 탐색, 자동 정밀도 선택은 구현 규모와 검증 비용이 큽니다. 현재 기능에 포함된 것으로 설명하지 않습니다.

예제와 해석

사용자 기준: 실제 이득이 작으면 유지하고, custom 복잡도를 늘리는 큰 변경은 필요성과 설계부터 검토합니다.

모델의 계산이 주소와 명령으로 구체화됩니다

각 단계의 결과를 다음 단계의 입력으로 읽으면 전체 흐름이 이어집니다.

동작을 따라 읽는 그림

단계	P = X × B를 따라가기	확인할 결과
모델 / Relax	P를 만들고 add/SiLU가 소비	Authored / graph prefix IR
Semantic TIR	C32 누산 → P를 FP16로 저장	Reduction과 cast 위치
Schedule	K 두 조각 / 입력 panel 재사용	Cache 위치와 intrinsic
Memory	P의 수명 / SRAM 보관 / 주소	실제 선택과 byte offset
ISA / runtime	LOAD·M_MUL·SAVE / 출력 해석	Program / plan / 실행 검증

01 IR에서 실행까지 같은 값을 추적

모델의 중간값 P가 call_tir 결과가 되고, TIR의 buffer가 되며, SRAM 주소와 SAVE/LOAD 명령으로 구체화됩니다. 단계가 바뀌어도 값의 의미는 연결됩니다.

02 결정과 증거를 함께 읽는다

왜 해당 fusion·schedule·보관을 골랐는지는 선택 진단으로, 실제 동작은 IR·명령·실행 결과로 확인합니다. 최적화 옵션이 켜졌다는 사실만으로 효과를 단정하지 않습니다.

03 질문에서 관련 단계로 돌아간다

반올림은 수치 정의, 반복 전송은 schedule, 중간값 보관은 memory, decode slot은 runtime 계약을 봅니다. 문서와 실험 파일이 같은 주제로 연결됩니다.

예제와 해석

다음 부록은 TVM API, IR, 기존 main, 양자화, 긴 prefill, 자료 추적을 정리합니다.

표준 API가 맡는 역할을 구분합니다

API 자체의 기능과 NPU가 추가로 결정하는 정책을 함께 봅니다.

동작을 따라 읽는 그림

표준 API	입력 → 결과	그 자체로 결정하지 않는 것
LegalizeOps / TE	고수준 연산 → 원소 수식	NPU의 허용 오차 정책
FuseOps / FuseTIR	Graph 그룹 → 하나의 PrimFunc	실제 장치에서의 이득
split / reorder	Loop 분할 / 의존성 내 순서 변경	최소 DMA 후보
cache / compute_at	복사 stage / loop 위치 변경	공간에 맞는 탐색 정책
tensorize	계산 조각 → 등록된 intrinsic	임의 연산의 ISA 구현
Memory passes	수명·충돌 → storage 배치	호출 간 보관과 persistent KV

01 TVM은 구조 변환의 기반

LegalizeOps와 TE는 계산 정의, FuseOps/FuseTIR는 graph grouping과 TIR 결합, Schedule API는 loop와 cache 변환을 제공합니다.

02 장치 정책은 그 위에 놓인다

NPU용 dtype·intrinsic·SRAM 용량·후보 비용은 backend가 정합니다. 표준 API를 사용한다는 것과 target별 판단이 필요 없다는 것은 다른 이야기입니다.

03 MetaSchedule은 선택형으로 연결

JSONDatabase는 선택된 trace를 저장·재생하고 ReplayFunc는 NPU schedule 설정을 추첨합니다. DLight·학습형 CostModel은 사용하지 않습니다. DMA를 latency로 기록하지 않습니다.

예제와 해석

StaticPlanBlockMemory와 USMP의 분석 범위도 다릅니다. 표준 pass의 이름보다 입력·출력·보장하는 내용을 먼저 확인합니다.

같은 matmul을 세 수준에서 읽습니다

아래는 구조를 단순화한 표현입니다. 실제 dump 파일은 발표자 노트에 연결했습니다.

동작을 따라 읽는 그림

Relax: 함수 호출과 결과 shape

```
p = R.call_tir(matmul, (x, B), out_sinfo=R.Tensor((64,64), "float16"))
```

↓

Semantic TIR: 원소의 정의

```
C32[i,j] += fp32(X[i,k]) * fp32(B[k,j])
P[i,j] = fp16(C32[i,j])
```

↓

Scheduled TIR: 복사와 tiled 실행 (간략한 의사 표현)

```
copy X/B panel to SRAM → for k_tile in range(2): npu_gemm_acc(...)
전체 K 완료 → FP16 SAVE → add / SiLU 소비
```

01 Relax에서 읽을 것

call_tir는 어떤 함수를 어떤 값에 적용하고 어떤 tensor를 받는지 표현합니다. 여전히 graph의 의존성과 함수 입출력이 보입니다.

02 Semantic TIR에서 읽을 것

Spatial/reduction 축, T.init, cast 위치를 봅니다. FP32 논리 buffer가 반드시 같은 크기의 SRAM 배열로 materialize되는 것은 아닙니다.

03 Scheduled TIR에서 읽을 것

Cache copy가 어느 loop 안에 있는지, match_buffer가 어떤 영역을 가리키는지, npu_gemm_acc가 어디에 있는지 봅니다. 최종 ISA는 다음 codegen 결과입니다.

예제와 해석

그림은 읽는 요점을 줄인 의사 표현입니다. 전체 실제 dump는 슬라이드 근거 링크의 .py 파일에 보존했습니다.

기존 main과 최신 v09는 실행 경계가 다릅니다

Main ver.08/source-0818은 commit 1143abb로 고정한 별도 최종 backend입니다.

동작을 따라 읽는 그림

항목	Main ver.08	현재 v09
Lowering	Relax 직접 emitter	TIR + Schedule + 공통 codegen
메모리	Flat G-buffer	DRAM ↔ SRAM ↔ unit
Layer	Host가 같은 layer program 반복	전체 entry stream / 판단 cache
Decode	Context 길이별 program	Capacity + mask / bucket
KV	Host concat	별도 출력 + NPU append tail

01 Main은 별도 최종 backend

Commit 1143abb의 ver.08은 Relax 직접 emitter와 flat G-buffer를 사용합니다. Host가 layer 프로그램을 반복하고 context별 decode를 준비합니다.

02 현재 v09는 계층과 TIR를 사용

TIR schedule을 거쳐 DRAM↔SRAM↔unit 명령을 만들고 전체 entry stream을 실행합니다. Decode는 capacity/mask와 append tail을 사용합니다.

03 숫자를 바로 배율로 비교하지 않음

컴파일 단위와 메모리 경계가 달라 같은 words 수가 같은 작업량을 뜻하지 않습니다. 기존 결과를 보존하되 현재 검증과 섞지 않습니다.

예제와 해석

두 backend를 혼합해 설명하지 않습니다. 기존 main의 동작과 실험은 docs/main 아래 별도 문서를 기준으로 합니다.

저장 dtype과 실제 연산 dtype을 구분해야 합니다

양자화는 별도 후반부 기능이며 전체 generation 검증은 아직 남아 있습니다.

동작을 따라 읽는 그림

W8A16

INT8 W + scale
DRAM 저장

→

FP16 W 복원
SRAM dequant

→

FP16 GEMM
FP32 accumulator

W8A8

INT8 W / 양자화 X
Per-channel / per-row

→

INT8 dot64
INT32 부분합

→

Scale 후 누산
FP32 → FP16 저장

01 W8A16: 저장을 줄이고 복원

Weight를 INT8와 output-channel scale로 저장합니다. SRAM에서 FP16으로 복원한 뒤 FP16 GEMM과 FP32 누산을 수행하므로 모든 연산이 INT8인 것은 아닙니다.

02 W8A8: 입력도 동적으로 양자화

현재 FP16 X를 행별 scale로 INT8화합니다. INT8 dot64의 INT32 부분합에 scale을 적용하고 FP32로 누산하여 FP16 결과를 만듭니다.

03 완료된 범위를 분리

작은 양자화 kernel 검증은 있으나 공개 generate 경로 연결과 전체 연속 생성 검증은 남아 있습니다. QK/PV·KV cache까지 전부 INT8이 된다고 해석하지 않습니다.

예제와 해석

Weight-only 준비는 host lifting 전에 rewrite하여 host로 옮깁니다. 본문의 28-layer 출력 검증은 FP16 경로입니다.

X와 W가 모두 커지면 양쪽을 나누어 올립니다

S=3072 gate projection의 설명용 산술입니다. 새 장문 compile 측정이 아닙니다.

동작을 따라 읽는 그림

전체 X: 18 MiB

64행 panel 384 KiB × 48개

전체 W: 48 MiB

256열 panel 1.5 MiB × 32개

바깥 loop	매 panel마다 다시 읽는 것	입력 전송 산술
X panel 48개	각 X panel에서 W 전체 순회	18 + 48×48 = 2322 MiB
W panel 32개	각 W panel에서 X 전체 순회	48 + 18×32 = 624 MiB

출력·scratch 비용 제외 / 설명용 비교 / 실제 장문 compile 아님

01 두 입력 모두 8 MiB보다 크다

S=3072일 때 X[3072,3072]는 18 MiB, gate W[3072,8192]는 48 MiB입니다. X 64행은 384 KiB, W 256열은 1.5 MiB로 나눌 수 있습니다.

02 바깥에 두는 쪽이 재사용을 정한다

X panel을 유지하며 W를 돌면 W 전체를 48번 읽고, W panel을 유지하며 X를 돌면 X 전체를 32번 읽습니다. 원래 tensor 크기가 같아도 DMA는 달라집니다.

03 설명용 산술과 실제 선택은 다르다

실제 탐색은 더 큰 panel과 다른 cache 위치, resident 입력을 검토하며 출력·scratch·누산 합법성도 검사합니다. 이 그림은 장문 전체 모델의 새 compile 결과가 아닙니다.

예제와 해석

입력 전송의 단순 산술: X 유지 2322 MiB, W 유지 624 MiB. 출력·scratch·정렬 비용을 뺀 비교이며 전역 최적을 뜻하지 않습니다.

발표의 그림과 숫자를 문서에서 다시 확인할 수 있습니다

슬라이드마다 발표자 노트와 원본 문서·실험 파일 경로가 있습니다.

동작을 따라 읽는 그림



파일	역할
slide_content.json	화면 설명·노트·근거의 편집 원본
build_deck.py	편집 가능한 도형과 PPTX/PDF 재생성
sources.json / validation.json	슬라이드별 출처 hash / 제작 검사 범위

01 발표→문서→실제 결과

각 슬라이드의 sources는 docs의 같은 주제 장과 IR·JSON으로 연결됩니다. 노트에는 설명 흐름과 조건을 남겼고 본문에도 핵심 이유와 예시를 담았습니다.

02 재생성 가능한 그림과 표

도형은 PPTX에서 편집할 수 있고, 수치 그림은 보존된 JSON을 기준으로 만듭니다. PDF는 같은 좌표에서 별도로 렌더링한 보기용 파일입니다.

03 근거 갱신을 혼동하지 않는다

발표자료를 다시 만들었다고 compiler나 전체 모델을 재실행한 것은 아닙니다. sources.json과 validation.json에서 원본 hash와 제작 검사 범위를 확인합니다.

예제와 해석

더 읽기: docs/README.md의 13개 장 → examples의 IR·명령 → evidence의 실행·선택·회귀 기록.